

LiQuID

Lindblad Quantum Integrated Dynamics

Version 0.7.0

User Guide Manual

A Comprehensive Reference for Physicists and Quantum Software Engineers

Monte Carlo Wave Function Simulation · Adaptive ODE Solvers · Sparse Operators · Parallel
Ensemble Management

C++17 · OpenMP · Header-Only Core

Compiled June 6, 2026

This document covers the complete public API, mathematical foundations, performance model, and extension seams of LiQuID v0.7.0.

Preface

LiQuID (*Lindblad Quantum Integrated Dynamics*) is a C++17 library for high-performance simulation of open quantum systems via the *Monte Carlo Wave Function* (MCWF) method. It is designed for research physicists, quantum optics experimentalists, and quantum software engineers who require both mathematical rigour and production-grade performance in a single, dependency-free framework.

What this manual covers. This guide documents every public class, function, and configuration parameter in LiQuID v0.7.0. It is organized in layers that mirror the library's own dependency graph: from foundational type aliases and the random number generator, through the linear algebra primitives and ODE solvers, to the complete simulation pipeline. Each section anchors code examples directly to the underlying mathematics and explains the memory management decisions that determine performance on modern hardware.

What this manual does not cover. Density-matrix (Lindblad master equation) solvers, GPU backends, and Python bindings are explicitly out of scope for v0.7.0. MCWF is the single computational paradigm of the library; this focus is intentional and is discussed in chapter 2.

How to use this manual. Readers new to the MCWF method should start with chapter 2 before examining the API. Experienced users may proceed directly to chapter 3. The performance chapter (chapter 12) should be consulted before deploying simulations with $N \geq 100$ or more than 10^4 trajectories.

Documentation Style Guide

This chapter defines the visual language used throughout the manual. All code examples, mathematical formulations, and callout blocks follow the conventions below without exception.

C++ Code Blocks

All C++ listings use `minted` with the `tango` style and the `cpp` lexer. Line numbers are shown for blocks exceeding ten lines. Identifiers are cross-referenced to their defining headers on first use.

Example of a fully annotated, non-abbreviated code block:

```
1  #include "liquid/liquid.hpp"
2  using namespace liquid;
3
4  // Build the system: two-level atom, decay rate gamma
5  DenseOperator make_sigma_minus(double gamma) {
6      DenseOperator L(2); // 2x2 zero-initialized
7      L(1, 0) = Scalar{std::sqrt(gamma), 0.0}; // sigma_- = |0><1|
8      return L;
9  }
```

Inline code references are typeset as `liquid::StateVector` or `apply_add()`.

Mathematical Formulations

All equations are typeset in display mode with `amsmath`. Quantum states use Dirac notation via the `physics` package. The Lindblad master equation is the central object and is always written as:

$$\frac{d\hat{\varrho}}{dt} = -i[H, \hat{\varrho}] + \sum_k \left(L_k \hat{\varrho} L_k^\dagger - \frac{1}{2} \{ L_k^\dagger L_k, \hat{\varrho} \} \right) \quad (1)$$

The effective non-Hermitian Hamiltonian used by the MCWF algorithm is:

$$H_{\text{eff}} = H - \frac{i}{2} \sum_k L_k^\dagger L_k \quad (2)$$

Callout Block Reference

A **NOTE** provides supplementary context, historical background, or a clarification of non-obvious behaviour.

A **WARNING** identifies a precondition that, if violated, produces incorrect results, undefined behaviour, or numerical failure. Always read warnings before using the associated API.

A **PERFORMANCE TIP** describes a measurable optimisation strategy derived from the memory layout or computational complexity analysis presented in chapter 12.

An **INVARIANT** is a mathematical or memory constraint that LiQuID guarantees at all times. Violating an invariant from outside the library constitutes undefined behaviour.

Contents

Preface	i
Documentation Style Guide	iii
I Foundations	1
1 Introduction and Architecture	3
1.1 Scope and Identity	3
1.2 Repository Structure	3
1.3 Build System and Requirements	4
1.3.1 Prerequisites	4
1.3.2 CMake Configuration	4
1.3.3 Linking Against LiQuID	5
2 Theoretical Foundations	7
2.1 The Lindblad Master Equation	7
2.1.1 Physical Interpretation of Jump Operators	7
2.2 The MCWF Stochastic Unravelling	8
2.2.1 Core Idea	8
2.2.2 The Effective Non-Hermitian Hamiltonian	8
2.2.3 Between-Jump Evolution	9
2.2.4 Jump Detection and Channel Selection	9
2.2.5 Observable Estimation	9
2.2.6 Algorithm Summary	10
II API Reference	11
3 Quick Start	13
3.1 The Five-Minute Example	13
3.2 Building the Example	14
4 Layer 0: Foundation	15
4.1 Type System	15
4.1.1 Scalar Types	15
4.1.2 The Imaginary Unit	15

4.1.3	Enumeration Types	16
4.2	Configuration Structs	16
4.2.1	PropagatorConfig	16
4.2.2	StoppingCriteria	17
4.3	Random Number Generator	17
4.3.1	Algorithm: xoshiro256**	17
4.3.2	Seeding Protocol	18
5	Layer 1: Linear Algebra	19
5.1	StateVector	19
5.1.1	Construction and Lifecycle	19
5.1.2	Element Access	19
5.1.3	Mathematical Operations	20
5.1.4	Normalization Invariant	20
5.2	DenseOperator	20
5.2.1	Storage Layout and Cache Behaviour	20
5.2.2	Construction	21
5.2.3	The <code>apply_add</code> Interface	21
5.2.4	Algebraic Operations (Cold Path)	21
5.3	SparseOperator	22
5.3.1	CSR Format	22
5.3.2	Construction from Triplets (COO $\rightarrow$ CSR)	22
5.3.3	The Sparse <code>apply_add</code> Interface	22
5.3.4	Conversion and Free Functions	23
6	Layer 1: ODE Solvers	25
6.1	The Stepper Policy	25
6.2	Fixed-Step RK4 (RK4Stepper)	26
6.2.1	Algorithm	26
6.2.2	Memory Layout	26
6.2.3	API	27
6.3	Adaptive DOPRI45 (DormandPrince45)	27
6.3.1	Algorithm	27
6.3.2	PI StepSize Controller	28
6.3.3	FSAL Implementation	28
6.3.4	Configuration	28
7	Layer 2: Quantum System	31
7.1	LindbladSet	31
7.1.1	Why Precompute Γ ?	31
7.1.2	Construction and Immutability	31
7.1.3	Hot-Path Interface	32
7.1.4	Cold-Path Interface	32
7.2	OpenSystem	33
7.2.1	Dense Time-Independent System	33
7.2.2	Precomputed H_{eff}	33
7.2.3	Sparse Time-Independent System	33
7.2.4	Time-Dependent Hamiltonian	34
8	Layer 3: Trajectory	35
8.1	TrajectoryState	35

8.1.1	CoreState	35
8.1.2	Factory Function	35
8.1.3	DiagnosticRecord	36
8.2	MCWFPropagator	36
8.2.1	Per-Instance Scratch Space	36
8.2.2	Direct Usage	37
8.2.3	Convenience: <code>run_to_completion</code>	38
9	Layer 4: Ensemble Management	39
9.1	ObservableAccumulator	39
9.1.1	Observable Definitions	39
9.2	ConvergenceMonitor	40
9.3	Adaptive Allocator and Policies	40
9.4	EnsembleManager	41
10	Layer 5: Simulation Interface	43
10.1	SimulationBuilder	43
10.1.1	Complete Builder Reference	43
10.1.2	Running and Inspecting Results	44
10.2	Parameter Sweep	44
III	Examples and Validation	47
11	Physics Examples	49
11.1	Example 1: Two-Level Spontaneous Decay	49
11.2	Example 2: Sparse Jaynes–Cummings Steady State	50
11.3	Example 2: Driven Two-Level System	51
11.4	Example 3: Cavity QED — Jaynes–Cummings Model	53
11.5	Example 4: Parameter Sweeps	57
11.6	Example 5: Convergence-Based Stopping	59
11.7	Example 6: Sparse Hilbert Spaces	61
IV	Performance and Extension	65
12	Performance Guide	67
12.1	Computational Complexity Model	67
12.2	Memory Hierarchy Considerations	68
12.3	Dense vs. Sparse Crossover	68
12.4	Multi-Threading Scaling	68
13	Extending LiQuID	71
13.1	Custom ODE Stepper	71
13.2	Custom Allocation Policy	72
14	Test Suite	73
14.1	Unit Tests	73
14.2	Validation Tests	74
15	Known Limitations and Roadmap	75
15.1	Known Limitations in v0.7.0	75

A Mathematical Reference	77
A.1 Lindblad vs. GKSL Notation	77
A.2 Standard Error of the Mean	77
B API Quick Reference	79
B.1 Builder Method Summary	79

List of Figures

1.1	LiQuID dependency graph. Arrows point from dependent to dependency. All dependencies are strictly unidirectional; no circular includes exist.	4
12.1	Theoretical speedup of sparse vs. dense matrix-vector product for the bosonic annihilation operator a (non-zero elements: $N - 1$). The crossover to practical benefit occurs around $N \approx 32$	68

List of Tables

1.1	Build requirements for LiQuID v0.7.0.	4
1.2	CMake build options.	5
2.1	Standard Lindblad jump operators for common dissipation channels.	8
4.1	Core type aliases. All are defined in <code>liquid/core/types.hpp</code>	15
4.2	<code>DiagnosticLevel</code> values and their space complexity.	16
5.1	<code>StateVector</code> mathematical interface.	20
6.1	<code>RK4Stepper</code> memory footprint.	27
8.1	<code>MCWFPropagator</code> per-instance scratch buffers.	36
12.1	Computational cost of each layer per trajectory. N = Hilbert dimension, K = jump operators, M = ODE steps, J = number of jumps.	67
14.1	Unit test executables and their scope.	73
14.2	Physics validation tests and their analytical references.	74
15.1	Known limitations and planned resolution phases.	75
B.1	Complete <code>SimulationBuilder</code> method reference.	79

Part I

Foundations

Chapter 1

Introduction and Architecture

1.1 Scope and Identity

LiQuID is not a general-purpose quantum computing toolkit. Its scope is precisely stated:

LiQuID provides production-grade Monte Carlo Wave Function simulation of Markovian open quantum systems described by the Lindblad master equation, with adaptive trajectory management, convergence-based stopping, and an extensible ensemble allocation policy.

The library deliberately excludes density-matrix evolution, non-Markovian dynamics, and circuit-model quantum computation. This focus ensures that every design decision — from the choice of `Scalar = std::complex<double>` to the FSAL property of the Dormand–Prince integrator — is motivated by a single, well-understood computational task.

1.2 Repository Structure

LiQuID is distributed as a single static library with a header-only core. The *only* compiled translation unit is `src/core/rng.cpp`, which implements the xoshiro256** random number generator.

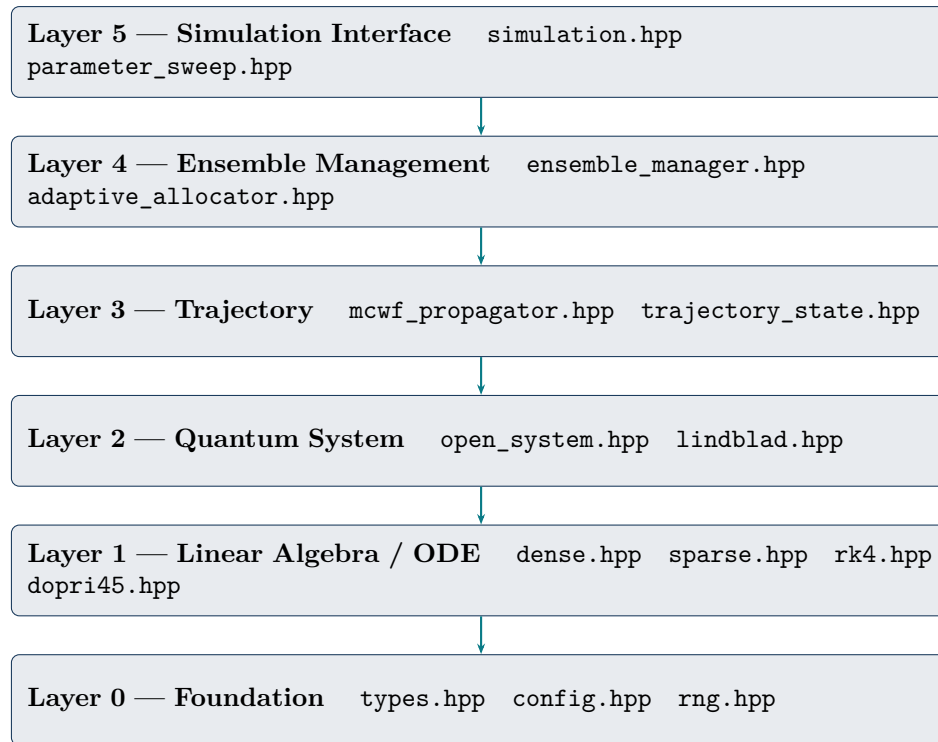


Figure 1.1: LiQuID dependency graph. Arrows point from dependent to dependency. All dependencies are strictly unidirectional; no circular includes exist.

1.3 Build System and Requirements

1.3.1 Prerequisites

Table 1.1: Build requirements for LiQuID v0.7.0.

Component	Version	Notes
CMake	≥ 3.16	Required for <code>target_compile_features</code>
C++ compiler	C++17	GCC ≥ 7 , Clang ≥ 5 , MSVC 2019+
OpenMP	Optional	Enables multi-threaded ensemble dispatch
Python + Pygments	Optional	Required only for <code>minted</code> -rendered docs

1.3.2 CMake Configuration

```

1  git clone https://github.com/your-org/LiQuID.git
2  cd LiQuID
3  cmake -B build -DCMAKE_BUILD_TYPE=Release \
4      -DLIQUID_OPENMP=ON \
5      -DLIQUID_TESTS=ON \
6      -DLIQUID_EXAMPLES=ON
7  cmake --build build --parallel
8  ctest --test-dir build --output-on-failure

```

CMake exposes four boolean options:

Table 1.2: CMake build options.

Option	Default	Effect
LIQUID_OPENMP	ON	Links OpenMP if found; falls back silently
LIQUID_TESTS	ON	Builds 10 unit + 7 validation test executables
LIQUID_EXAMPLES	ON	Builds <code>jc_steady_state</code> and <code>single</code>
LIQUID_SANITIZE	OFF	Enables AddressSanitizer + UBSan (Debug only)

1.3.3 Linking Against LiQuID

```
find_package(LiQuID REQUIRED)
target_link_libraries(my_simulation PRIVATE liquid)
```

Alternatively, add as a subdirectory:

```
add_subdirectory(extern/LiQuID)
target_link_libraries(my_simulation PRIVATE liquid)
```

The umbrella header `liquid/liquid.hpp` includes all public headers in correct dependency order. For large projects, you may include individual layer headers to reduce compilation time.

Chapter 2

Theoretical Foundations

2.1 The Lindblad Master Equation

An open quantum system is one that interacts with an unmonitored environment (reservoir). When the system–reservoir coupling is weak (*Born approximation*) and the reservoir correlation time is much shorter than the system dynamics (*Markov approximation*), the reduced density matrix $\hat{\rho}$ of the system evolves under the *Gorini–Kossakowski–Sudarshan–Lindblad* (GKSL) master equation:

$$\frac{d\hat{\rho}}{dt} = -i[H, \hat{\rho}] + \sum_{k=1}^K \left(L_k \hat{\rho} L_k^\dagger - \frac{1}{2} L_k^\dagger L_k \hat{\rho} - \frac{1}{2} \hat{\rho} L_k^\dagger L_k \right) \quad (2.1)$$

where:

- $H \in \mathbb{C}^{N \times N}$ is the Hermitian system Hamiltonian (*physical units*: $\hbar = 1$);
- $\{L_k\}_{k=1}^K$ are the *Lindblad jump operators*, which encode the dissipative channels of the system–environment interaction;
- $[\cdot, \cdot]$ denotes the commutator.

eq. (2.1) is the most general Markovian master equation that preserves the trace, Hermiticity, and positive semi-definiteness of $\hat{\rho}$.

LiQuID works in natural units $\hbar = 1$. All energies and rates are expressed in the same units as the Hamiltonian matrix elements. Concretely, if H_{00} has units of MHz, then decay rates γ_k must also be in MHz, and t_{final} in μs^{-1} .

2.1.1 Physical Interpretation of Jump Operators

Each operator L_k describes one incoherent process. Common examples in quantum optics are:

Table 2.1: Standard Lindblad jump operators for common dissipation channels.

Process	Operator	C++ construction
Spontaneous emission (rate γ)	$L = \sqrt{\gamma} \sigma_-$	<code>L(1,0) = Scalar{sqrt(gamma), 0};</code>
Cavity photon loss (rate κ)	$L = \sqrt{\kappa} a$	Lower-triangular with $\sqrt{n}$ entries
Pure dephasing (rate γ_ϕ)	$L = \sqrt{\gamma_\phi} \sigma_z$	Diagonal: $\pm\sqrt{\gamma_\phi}$
Thermal excitation (rate $\gamma\bar{n}$)	$L = \sqrt{\gamma\bar{n}} \sigma_+$	<code>L(0,1) = Scalar{sqrt(gamma_n), 0};</code>

2.2 The MCWF Stochastic Unravelling

2.2.1 Core Idea

The MCWF method (Dalibard, Castin and Mølmer, 1992; Dum, Zoller and Ritsch, 1992) *unravels* the master equation into an ensemble of pure-state trajectories. Each trajectory is a stochastic process on the Hilbert space $\mathcal{H} = \mathbb{C}^N$, and ensemble averages reproduce the density matrix:

$$\hat{\rho}(t) = \overline{|\psi(t)\rangle \langle \psi(t)|} \quad (2.2)$$

where the overline denotes an average over many independent realizations of the stochastic process. The computational advantage over direct master-equation integration is memory: storing $|\psi\rangle \in \mathbb{C}^N$ requires $\mathcal{O}(N)$ complex numbers, whereas $\hat{\rho}$ requires $\mathcal{O}(N^2)$. For a 100-photon Fock space ($N = 101$), this is the difference between ~ 1.6 kB and ~ 160 kB. For $N = 1000$, the ratio grows to $\times 1000$.

2.2.2 The Effective Non-Hermitian Hamiltonian

Define the aggregate decay operator:

$$\Gamma \equiv \sum_{k=1}^K L_k^\dagger L_k \quad (2.3)$$

and the effective Hamiltonian:

$$H_{\text{eff}} \equiv H - \frac{i}{2} \Gamma = H - \frac{i}{2} \sum_k L_k^\dagger L_k \quad (2.4)$$

H_{eff} is *not Hermitian*: $H_{\text{eff}} \neq H_{\text{eff}}^\dagger$. Its non-Hermitian part $-\frac{i}{2}\Gamma$ causes the norm of $|\psi\rangle$ to decrease over time, encoding the probability of *no jump having occurred* up to time t .

INV- Γ : In LiQuID, $\Gamma = \sum_k L_k^\dagger L_k$ is precomputed and stored immutably in `LindbladSet::gamma_` at construction time. It is never recomputed. Modifying the jump operators after construction is not possible because `LindbladSet::ops_` is private and the copy constructor is deleted.

2.2.3 Between-Jump Evolution

Between quantum jumps, the wavefunction satisfies the *non-Hermitian Schrödinger equation*:

$$\frac{d|\psi\rangle}{dt} = -i H_{\text{eff}} |\psi\rangle \quad (2.5)$$

This is integrated numerically by the ODE solver. The norm $\| |\psi(t)\rangle \|^2 = \langle \psi(t) | \psi(t) \rangle$ decreases monotonically between jumps. Specifically, at time t after the last normalization event:

$$\frac{d}{dt} \| |\psi\rangle \|^2 = -\langle \psi | \Gamma | \psi \rangle = -\sum_k \| L_k |\psi\rangle \|^2 \leq 0 \quad (2.6)$$

2.2.4 Jump Detection and Channel Selection

A random threshold $r \sim \text{Uniform}(0, 1]$ is drawn at the start of each inter-jump interval (and redrawn immediately after every jump). A jump is detected when:

$$\| |\psi(t)\rangle \|^2 < r \quad (2.7)$$

Upon detection, the jump channel k^* is selected with probability proportional to $\| L_k |\psi\rangle \|^2$:

$$\text{Pr}(\text{channel } k) = \frac{\| L_k |\psi\rangle \|^2}{\sum_j \| L_j |\psi\rangle \|^2} \quad (2.8)$$

The post-jump state is renormalized:

$$|\psi\rangle \leftarrow \frac{L_{k^*} |\psi\rangle}{\| L_{k^*} |\psi\rangle \|} \quad (2.9)$$

2.2.5 Observable Estimation

For a Hermitian observable O , the MCWF estimator is:

$$\langle O \rangle(t) \approx \frac{1}{M} \sum_{m=1}^M \frac{\langle \psi^{(m)}(t) | O | \psi^{(m)}(t) \rangle}{\langle \psi^{(m)}(t) | \psi^{(m)}(t) \rangle} \quad (2.10)$$

where the denominator accounts for the fact that trajectories are stored in an unnormalized form between output intervals. In LiQuID, the `ObservableFn` callable is responsible for the normalization; the `ObservableAccumulator` computes only the sum and variance.

2.2.6 Algorithm Summary

Algorithm 2.1 (H). Algorithm: MCWF Single Trajectory (one call to `MCWFPropagator::advance`)

1. Select step size δt (warm-start from previous step).
2. **ODE step**: integrate eq. (2.5) from t to $t + \delta t$ using the configured stepper (RK4 or DOPRI45), yielding $|\psi_{\text{prop}}\rangle$.
3. **Jump check**: if $\| |\psi_{\text{prop}}\rangle \|^2 > r$:
 - 3a. Accept step: $|\psi\rangle \leftarrow |\psi_{\text{prop}}\rangle$, $t \leftarrow t + \delta t$.
 - 3b. Return `Advanced` (or `Completed` if $t \geq t_{\text{final}}$).
4. **Jump handling**: a jump occurred in $[t, t + \delta t]$:
 - 4a. (Phase 3) Locate exact jump time via bisection.
 - 4b. Compute $p_k = \|L_k |\psi\rangle\|^2$ for all k .
 - 4c. Draw channel $k^* \sim \{p_k / \sum p_k\}$.
 - 4d. Apply: $|\psi\rangle \leftarrow L_{k^*} |\psi\rangle / \|L_{k^*} |\psi\rangle\|$.
 - 4e. Redraw $r \sim \text{Uniform}(0, 1]$.
 - 4f. Reset stepper (state is discontinuous).
 - 4g. Return `JumpOccurred`.
5. If numerical failure (step size collapse): return `Failed`.

`advance()` returns at the *first* jump, not after all jumps to t_{target} . This event-driven design allows the `EnsembleManager` to inspect, suspend, or redirect a trajectory after each jump without re-entering the propagation loop.

Part II

API Reference

Chapter 3

Quick Start

3.1 The Five-Minute Example

The following complete program simulates a two-level atom decaying from its excited state at rate $\gamma = 1.0$, with a resonance frequency $\omega = 1.0$. The exact solution for the expectation value of $\sigma_z = |e\rangle\langle e| - |g\rangle\langle g|$ is:

$$\langle\sigma_z(t)\rangle = 2e^{-\gamma t} - 1 \quad (3.1)$$

```
1  #include "liquid/liquid.hpp"
2  using namespace liquid;
3  using namespace liquid::ensemble;
4
5  int main() {
6      // -- Physical parameters -----
7      const double omega = 1.0;    // transition frequency [arbitrary units]
8      const double gamma = 1.0;    // spontaneous emission rate
9      const double T      = 3.0;    // total simulation time
10
11     // -- Hamiltonian: H = (omega/2) * sigma_z -----
12     // Basis: |0> = excited, |1> = ground
13     DenseOperator H(2);
14     H(0, 0) = Scalar{ omega / 2.0, 0.0};
15     H(1, 1) = Scalar{-omega / 2.0, 0.0};
16
17     // -- Jump operator: L = sqrt(gamma) * sigma_minus -----
18     // sigma_minus = |1><0|: maps excited |0> to ground |1>
19     DenseOperator L(2);
20     L(1, 0) = Scalar{std::sqrt(gamma), 0.0};
21
22     // -- Observable: sigma_z = |0><0| - |1><1| -----
23     DenseOperator sz(2);
24     sz(0, 0) = Scalar{ 1.0, 0.0};
25     sz(1, 1) = Scalar{-1.0, 0.0};
26
```

```

27 // -- Build simulation -----
28 Simulation sim = SimulationBuilder{
29     .hamiltonian(std::move(H))
30     .collapse_operator(std::move(L))
31     .observe("sigma_z", std::move(sz))
32     .seed(42)
33     .dt(1e-3)
34     .stop_at_sem(0.01)           // stop when relative SEM < 1%
35     .min_trajectories(100)
36     .max_trajectories(10000)
37     .build();
38
39 // -- Initial state: excited state |0> -----
40 StateVector psi0(2);
41 psi0[0] = Scalar{1.0, 0.0};    // |0> component
42 psi0[1] = Scalar{0.0, 0.0};    // |1> component
43
44 // -- Run -----
45 auto result = sim.run(psi0, 0.0, T);
46
47 // -- Output -----
48 const double exact = 2.0 * std::exp(-gamma * T) - 1.0;
49 std::printf("<sigma_z>(T=%.1f) = %.4f +/- %.4f [exact: %.4f] N=%zu\n",
50     T,
51     result.observables[0].mean,
52     result.observables[0].sem,
53     exact,
54     result.total_trajectories);
55
56 return 0;
57 }

```

Expected output (exact value ≈ -0.9003):

```
<sigma_z>(T=3.0) = -0.9011 +/- 0.0089 [exact: -0.9003] N=247
```

The simulation terminates at $N = 247$ because the relative standard error fell below the 1% target before reaching the maximum trajectory count. The minimum count of 100 was already satisfied.

3.2 Building the Example

```

g++ -std=c++17 -O3 -march=native -DNDEBUG \
-I path/to/LiQuID/include \
-L path/to/LiQuID/build \
-o two_level_decay two_level_decay.cpp \
-lliquid

```

Chapter 4

Layer 0: Foundation

4.1 Type System

All fundamental types are declared in `liquid/core/types.hpp` and are available without qualification under namespace `liquid`.

4.1.1 Scalar Types

Table 4.1: Core type aliases. All are defined in `liquid/core/types.hpp`.

Alias	Underlying type	Semantic
Scalar	<code>std::complex<double></code>	Complex quantum amplitude
Real	<code>double</code>	Real-valued quantity (time, rate, norm)
Dim	<code>std::size_t</code>	Hilbert space dimension N
Idx	<code>std::size_t</code>	Row/column matrix index
TrajId	<code>std::uint64_t</code>	Unique trajectory identifier
Seed	<code>std::uint64_t</code>	RNG seed

`Scalar = std::complex<double>` is a **frozen design decision**. The library is not templated on the floating-point type. Single-precision (`float`) and GPU (`half`) support are explicitly deferred to a future policy layer and will not appear in the public API before v1.0.

4.1.2 The Imaginary Unit

```
// In liquid/core/types.hpp:  
inline constexpr Scalar i_unit{0.0, 1.0};
```

Use `i_unit` in preference to `std::complex<double>(0,1)` or the raw literal `1i`. The named constant is self-documenting and avoids the UDL `_i` suffix's namespace dependency.

```
// Correct usage:
using namespace liquid;
Scalar z = 2.5 * i_unit;           // z = 0 + 2.5i
Scalar w = i_unit * H_element;    // w = i * h

// Equivalent but less idiomatic:
Scalar z2 = Scalar{0.0, 2.5};
```

4.1.3 Enumeration Types

DiagnosticLevel Controls the volume of per-trajectory diagnostic data retained in memory:

Table 4.2: DiagnosticLevel values and their space complexity.

Value	Space	Content
None	$\mathcal{O}(1)$	No diagnostics stored; zero per-step overhead
Summary	$\mathcal{O}(N_j)$	Summary statistics + jump event history (default)
Full	$\mathcal{O}(N_t)$	Above + subsampled norm history
StepHistory	$\mathcal{O}(N_t)$	All of the above + per-step ODE records (expensive)

TrajectoryStatus Trajectory lifecycle states: `Initialized` $\rightarrow$ `Running` $\rightarrow$ `{Completed | Failed}`. The `Suspended` state is used by the `EnsembleManager` for adaptive allocation (see section 9.3).

SolverType `RK4Fixed` (Phase 1) and `DormandPrince45` (Phase 3). Select via `SimulationBuilder::solver()`.

4.2 Configuration Structs

4.2.1 PropagatorConfig

Controls the numerical behaviour of `MCWFPropagator`. All fields have defaults calibrated for quantum optics problems with $\gamma \sim \omega \sim 1$.

```
1 struct PropagatorConfig {
2     // ODE stepping
3     Real dt_initial      = 1e-3;    // Initial stepsize
4     Real dt_min          = 1e-12;   // Floor: triggers Failed status if hit
5     Real dt_max          = 1e-1;    // Ceiling: prevents overshooting features
6
7     // Adaptive solver tolerances (used by DormandPrince45)
8     Real atol            = 1e-8;    // Absolute local error tolerance
9     Real rtol            = 1e-6;    // Relative local error tolerance
10
11     // Jump detection
12     Real jump_tol        = 1e-6;    // Accuracy of bisection jump location
13     int jump_bisect_max  = 50;      // Max bisection iterations
```

```

14     int max_step_rejects = 100;    // Max DOPRI45 rejections before Failed
15 };

```

Setting `dt_min` too large will cause the solver to declare Failed for stiff problems. If you observe unexpected failures, lower `dt_min` before increasing solver tolerances.

4.2.2 StoppingCriteria

```

1  struct StoppingCriteria {
2      Real      target_rel_sem      = 1e-2;    // 1% relative SEM
3      std::size_t min_trajectories   = 100;     // Never stop before this
4      std::size_t max_trajectories   = 100000;  // Hard ceiling
5      Real      max_wall_seconds     = 0.0;     // 0 = no limit
6      bool      enforce_min         = true;     // Respect min even if converged
7  };

```

The monitor evaluates the following decision hierarchy after each batch:

1. If $N < N_{\min}$: Continue (hard floor).
2. If $\max_i \text{SEM}_i / |\mu_i| < \varepsilon_{\text{rel}}$: Converged.
3. If $N \geq N_{\max}$ or elapsed time $> t_{\text{wall}}$: BudgetHit.
4. If `request_stop()` was called: Forced.
5. Otherwise: Continue.

4.3 Random Number Generator

4.3.1 Algorithm: xoshiro256**

`RNGState` implements the *xoshiro256*** algorithm by Blackman and Vigna (2018). Key properties:

- 256-bit state (four `uint64_t`): 32 bytes.
- Period $2^{256} - 1 \approx 1.16 \times 10^{77}$.
- Passes all BigCrush and PractRand statistical tests.
- Approximately $4\times$ faster than `std::mt19937_64` with $78\times$ smaller state.
- *No heap allocation*. The entire state fits in two cache lines.

4.3.2 Seeding Protocol

```

1  class RNGState {
2  public:
3      // Combine global_seed and traj_id via splitmix64.
4      // Different (global_seed, traj_id) pairs are guaranteed to
5      // produce statistically independent streams.
6      void seed(Seed global_seed, TrajId traj_id) noexcept;
7
8      // Returns a value in (0, 1] -- note: open at zero.
9      // Zero is excluded to prevent spurious immediate jumps at t=0.
10     Real draw_uniform() noexcept;
11
12     // Returns an integer in [0, n-1].
13     std::size_t draw_int(std::size_t n) noexcept;
14
15     // State serialization for checkpointing.
16     void serialize(std::ostream& os) const;
17     static RNGState deserialize(std::istream& is);
18 };

```

INV-3 (RNG Exclusivity): `core.rng` is the *sole* source of randomness for its trajectory. No global, static, or thread-local RNG is accessed anywhere in LiQuID. This guarantees that simulation results are byte-identical regardless of thread scheduling, provided the seed and trajectory count are the same.

The seeding protocol uses *splitmix64* — a bijective hash on `uint64_t` — to combine the global seed and trajectory identifier:

$$s_0^{(m)} = \text{splitmix64}(g \oplus m) \quad (4.1)$$

where g is the global seed and m is the trajectory index. Different values of g produce statistically independent ensembles, enabling reproducible parameter sweeps.

Chapter 5

Layer 1: Linear Algebra

5.1 StateVector

`StateVector` represents a quantum state $|\psi\rangle \in \mathbb{C}^N$. It is stored as a contiguous `std::vector<Scalar>` — a flat array of N complex doubles on the heap.

5.1.1 Construction and Lifecycle

```
1  // Construct zero-initialized vector of dimension N
2  StateVector psi(N);
3
4  // Construct with explicit fill value
5  StateVector psi(N, Scalar{0.5, 0.0});
6
7  // Copy and move semantics follow standard value semantics.
8  // Copies are deep (new allocation); moves are O(1).
9  StateVector phi = psi;           // deep copy
10 StateVector chi = std::move(psi); // O(1) move; psi is now empty
```

5.1.2 Element Access

```
psi[0] = Scalar{1.0, 0.0}; // excited state amplitude
psi[1] = Scalar{0.0, 0.0}; // ground state amplitude

// Bounds-checking in Debug builds (assert); unchecked in Release.
// Idx is std::size_t -- never pass negative integers.
```

5.1.3 Mathematical Operations

Table 5.1: StateVector mathematical interface.

Method signature	Alloc?	Computes
Real norm_sq() const	No	$\sum_i \psi_i ^2$
Real norm() const	No	$\sqrt{\sum_i \psi_i ^2}$
void normalize()	No	$ \psi\rangle \leftarrow \psi\rangle / \ \psi\rangle \ $
void scale(Scalar a)	No	$ \psi\rangle \leftarrow \alpha \psi\rangle$
void add_scaled(sv, a)	No	$ \psi\rangle += \alpha \phi\rangle$
void set_zero()	No	$ \psi\rangle \leftarrow \mathbf{0}$
void copy_from(sv)	No	$ \psi\rangle \leftarrow \phi\rangle$
static Scalar inner(a,b)	No	$\langle a b\rangle = \sum_i \bar{a}_i b_i$

`add_scaled()` and `copy_from()` are the two highest-frequency operations inside every ODE stepper. They are implemented as tight for loops over the contiguous data array. On x86-64 with `-O3 -march=native`, the compiler auto-vectorizes them using AVX2 (4 complex doubles per SIMD register). Calling `apply()` (which allocates) inside an ODE loop defeats this optimization.

5.1.4 Normalization Invariant

INV-1 (Normalization): $\| |\psi\rangle \|^2 \approx 1.0$ at $t = t_0$ and immediately after every jump. Between jumps, $r < \| |\psi\rangle \|^2 \leq 1.0$ (strictly decreasing per eq. (2.6)). `normalize()` asserts `norm() > 0` in Debug builds and is undefined behaviour if called on the zero vector in Release.

5.2 DenseOperator

`DenseOperator` represents an $N \times N$ complex matrix $O \in \mathbb{C}^{N \times N}$. It is stored *row-major*: element (i, j) maps to `data_[i * dim_ + j]`.

5.2.1 Storage Layout and Cache Behaviour

Row-major storage is chosen because the dominant operation — `apply_add(psi, alpha, out)` — sweeps the matrix row by row, accumulating one output component per row. For row i :

$$\text{out}[i] += \alpha \sum_{j=0}^{N-1} \underbrace{A[i \cdot N + j]}_{\text{contiguous in cache}} \cdot \psi[j] \quad (5.1)$$

The inner sum reads N contiguous `Scalar` values from `data_`, which fits in $N \cdot 16$ bytes. For $N = 64$, row i occupies 1 kB — exactly one 16-way set of cache lines on a typical L1 cache.

5.2.2 Construction

```

1 // Zero-initialized  $N \times N$  matrix
2 DenseOperator H(N);
3
4 // From flat row-major data (no copy if moved)
5 std::vector<Scalar> flat_data(N * N, Scalar{0.0, 0.0});
6 flat_data[0 * N + 0] = Scalar{0.5, 0.0}; //  $H(0,0)$ 
7 flat_data[1 * N + 1] = Scalar{-0.5, 0.0}; //  $H(1,1)$ 
8 DenseOperator H2(N, std::move(flat_data));
9
10 // Factory functions
11 DenseOperator I = make_identity(N); //  $N \times N$  identity
12 DenseOperator Z = make_zero_operator(N); //  $N \times N$  zero
13 DenseOperator LL = adjoint_times(L); //  $LdagL$  (allocates)

```

5.2.3 The apply_add Interface

```

1 // HOT PATH -- no allocation, called at every ODE sub-step.
2 // Computes:  $out += \alpha * A * \psi$ 
3 // Preconditions:  $\psi.size() == dim\_$ ,  $out.size() == dim\_$ 
4 void DenseOperator::apply_add(
5     const StateVector& psi, // input state
6     Scalar alpha, // scaling factor
7     StateVector& out) // accumulator (modified in-place)
8     const noexcept;
9
10 // COLD PATH -- allocates output vector.
11 // Use only outside the ODE loop (operator testing, validation).
12 StateVector DenseOperator::apply(const StateVector& psi) const;

```

Never call `apply()` inside an ODE callback. It allocates a new `StateVector` on every invocation. For 10^4 trajectories with 10^3 steps each, this produces 10^7 allocations that defeat the allocator's free-list and fragment the heap. Always use `apply_add()` with a pre-allocated output vector.

5.2.4 Algebraic Operations (Cold Path)

```

1 DenseOperator Adag = A.adjoint(); // Adag -- allocates  $N \times N$ 
2 DenseOperator C = A.matmul(B); //  $A*B$  -- allocates  $N \times N$ 
3 A.add_scaled(B, Scalar{2.0, 0.0}); //  $A += 2*B$  (in-place)
4
5 // Free functions
6 Scalar ev = expectation(0, psi); //  $\langle \psi | O | \psi \rangle$  (allocates temp)

```

5.3 SparseOperator

`SparseOperator` stores an $N \times N$ sparse complex matrix in *Compressed Sparse Row* (CSR) format. It is intended for large Hilbert spaces where most matrix elements are structurally zero.

5.3.1 CSR Format

Three arrays define the sparse matrix:

$$\text{values}[k] \in \mathbb{C} \quad \text{value of the } k\text{-th stored element} \quad (5.2)$$

$$\text{col_indices}[k] \in [0, N) \quad \text{column of the } k\text{-th stored element} \quad (5.3)$$

$$\text{row_ptr}[i] \in [0, \text{nnz}] \quad \text{index in values where row } i \text{ begins} \quad (5.4)$$

Row i contains elements at columns $\text{col_indices}[\text{row_ptr}[i] : \text{row_ptr}[i + 1]]$ with values $\text{values}[\text{row_ptr}[i] : \text{row_ptr}[i + 1]]$. The array `row_ptr` has length $N + 1$; the sentinel `row_ptr[N] = nnz` gives the total non-zero count.

5.3.2 Construction from Triplets (COO $\rightarrow$ CSR)

```

1  // Triplet: (row, col, value)
2  std::vector<Triplet> trips;
3  trips.push_back({0, 0, Scalar{ 1.0, 0.0}}); // diagonal: A(0,0) = 1
4  trips.push_back({1, 1, Scalar{-1.0, 0.0}}); // diagonal: A(1,1) = -1
5  trips.push_back({1, 0, Scalar{ 0.5, 0.0}}); // off-diag: A(1,0) = 0.5
6
7  // Duplicate (row,col) entries are automatically summed.
8  // Elements with |value| < 1e-300 are dropped.
9  SparseOperator A(N, std::move(trips));
10
11 std::printf("nnz = %zu, sparsity = %.4f\n",
12             A.nnz(), A.sparsity()); // sparsity = nnz / (N*N)

```

After construction, `SparseOperator` is immutable with respect to its sparsity pattern. `add_scaled(B, alpha)` internally rebuilds the CSR structure from scratch. It is a cold-path operation intended for precomputation, not for use inside solvers.

5.3.3 The Sparse apply_add Interface

The hot-path interface is identical to `DenseOperator`:

```

1  // out += alpha * A * psi (no allocation)
2  // Inner loop exploits CSR structure: only nnz multiplications
3  // instead of N^2 for a dense matrix.
4  void SparseOperator::apply_add(

```

```

5      const StateVector& psi,
6      Scalar           alpha,
7      StateVector&      out) const noexcept;

```

The complexity of a sparse matrix-vector product is $\mathcal{O}(\text{nnz})$ rather than $\mathcal{O}(N^2)$. For the bosonic ladder operator a in an N -dimensional Fock space, $\text{nnz} = N - 1$, giving a $N/(N - 1) \approx N$ speedup over the dense case.

For Hilbert spaces with $N \leq 64$ and fully connected operators, use `DenseOperator`: cache reuse dominates and the overhead of indirect addressing in CSR is not worth the reduced operation count. Switch to `SparseOperator` when $\text{nnz}/N^2 \lesssim 0.3$, or equivalently when fewer than 30% of matrix elements are non-zero.

5.3.4 Conversion and Free Functions

```

1      // Convert dense to sparse (drops elements below zero_tol = 1e-14)
2      SparseOperator As = to_sparse(A_dense, 1e-14);
3
4      // Convert sparse to dense (allocates N*N array)
5      DenseOperator Ad = A_sparse.to_dense();
6
7      // Create sparse identity
8      SparseOperator I = make_sparse_identity(N);
9
10     // Compute AdagA for sparse operator
11     SparseOperator LdagL = sparse_adjoint_times(L);
12
13     // Expectation value <psi|O|psi>
14     Scalar ev = sparse_expectation(O_sparse, psi);

```


Layer 1: ODE Solvers

6.1 The Stepper Policy

Both ODE solvers satisfy the *Stepper named requirement* (C++17 *policy class* idiom). Any class satisfying this requirement can be used as the `StepperType` template parameter of `MCWFPropagator`. The requirement is documented below as an annotated struct; enforcement is via `static_assert` on the return type of `try_step` inside `MCWFPropagator`'s constructor.

```

1  // -- Stepper named requirement (C++17 policy idiom) -----
2  //
3  // A type S satisfies Stepper if it provides:
4  //
5  //   (1) StepResult try_step(Real t,
6  //                           const StateVector& psi_in,
7  //                           StateVector&      psi_out,
8  //                           Real               dt_suggested,
9  //                           Real               t_max,
10 //                           RHSFn&&          rhs_fn);
11 //
12 //   Integrates one step from (t, psi_in), writing the result into
13 //   psi_out. May shorten the step to respect t_max.
14 //   Returns a StepResult describing acceptance and the next dt hint.
15 //   RHSFn is any callable with signature:
16 //       void(Real t, const StateVector& in, StateVector& accum)
17 //
18 //   (2) void reset(Real dt_initial) noexcept;
19 //
20 //   Discards all internal stepper state (e.g. FSAL buffers) and
21 //   resets the working step size. Must be called after every
22 //   quantum jump because the state vector is discontinuous.
23 //
24 // The library enforces (1) at instantiation time:
25 //   static_assert(
26 //       std::is_same_v<
27 //           decltype(std::declval<S>().try_step(...)),
28 //           ode::StepResult>,

```

```

29 //      "StepperType must return ode::StepResult from try_step()";
30
31 struct StepResult {
32     bool accepted;      // true = step accepted; psi_out is valid
33                        // false = step rejected (DOPRI45 only); psi_out
34                        //      ↪ undefined
35     Real dt_taken;      // actual step length used
36     Real dt_next;      // suggested step length for the next call
37     Real local_err;    // normalised local error estimate (0.0 for RK4)
38 };

```

The RHS callable passed to `try_step` uses the *accumulator pattern* — it adds to its output buffer rather than overwriting it, so the stepper can zero the buffer once and reuse it across stages:

```

// Signature of the RHS callable accepted by every Stepper.
// Computes: accum += f(t, psi)  -- note: += not =
// The stepper calls set_zero() on accum before the first stage;
// rhs_fn must never zero it internally.
void rhs_fn(Real t, const StateVector& psi, StateVector& accum);

```

6.2 Fixed-Step RK4 (RK4Stepper)

6.2.1 Algorithm

The classical fourth-order Runge–Kutta formula integrates $d|\psi\rangle/dt = f(t, |\psi\rangle)$ over one step h :

$$k_1 = f(t, |\psi\rangle) \quad (6.1)$$

$$k_2 = f(t + h/2, |\psi\rangle + (h/2) k_1) \quad (6.2)$$

$$k_3 = f(t + h/2, |\psi\rangle + (h/2) k_2) \quad (6.3)$$

$$k_4 = f(t + h, |\psi\rangle + h k_3) \quad (6.4)$$

$$|\psi(t + h)\rangle = |\psi\rangle + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (6.5)$$

The local truncation error is $\mathcal{O}(h^5)$; the global error accumulates as $\mathcal{O}(h^4)$.

6.2.2 Memory Layout

`RK4Stepper` pre-allocates five `StateVector` buffers (k_1, k_2, k_3, k_4 , and a temporary $|\psi_{\text{tmp}}\rangle$) on the *first* call to `try_step()`, via `ensure_scratch(Dim n)`. Subsequent calls reuse them.

Table 6.1: RK4Stepper memory footprint.

Buffer	Size	Purpose
k1_	16N bytes	Stage 1 derivative
k2_	16N bytes	Stage 2 derivative
k3_	16N bytes	Stage 3 derivative
k4_	16N bytes	Stage 4 derivative
tmp_	16N bytes	Intermediate state
Total	80N bytes	(e.g., $80 \times 64 = 5120$ bytes for $N = 64$)

6.2.3 API

```

1  // Construct with fixed stepsize
2  ode::RK4Stepper stepper(1e-3);    // dt = 1e-3
3
4  // Or default-construct and configure via PropagatorConfig
5  ode::RK4Stepper stepper;
6  stepper.reset(1e-3);
7
8  // StepResult (always accepted for RK4)
9  struct StepResult {
10     bool   accepted;    // true always for RK4
11     Real   dt_taken;    // actual dt (may be clamped by t_max)
12     Real   dt_next;     // suggestion for next step (same as dt_taken for RK4)
13     Real   local_err;   // 0.0 for fixed-step
14 };

```

6.3 Adaptive DOPRI45 (DormandPrince45)

6.3.1 Algorithm

The Dormand–Prince 4(5) method is an embedded Runge–Kutta pair with 7 stage evaluations per accepted step. The Butcher tableau is (Hairer, Nørsett & Wanner, Table 5.2):

$$\begin{array}{c|cccccc}
 0 & & & & & & \\
 \frac{1}{5} & \frac{1}{5} & & & & & \\
 \frac{3}{10} & \frac{3}{40} & \frac{9}{40} & & & & \\
 \frac{4}{5} & \frac{44}{45} & -\frac{56}{15} & \frac{32}{9} & & & \\
 \frac{8}{9} & \frac{19372}{6561} & -\frac{25360}{2187} & \frac{64448}{6561} & -\frac{212}{729} & & \\
 1 & \frac{9017}{3168} & -\frac{355}{33} & \frac{46732}{5247} & \frac{49}{176} & -\frac{5103}{18656} & \\
 1 & \frac{35}{384} & 0 & \frac{500}{1113} & \frac{125}{192} & -\frac{2187}{6784} & \frac{11}{84} \\
 \hline
 \text{5th order} & \frac{35}{384} & 0 & \frac{500}{1113} & \frac{125}{192} & -\frac{2187}{6784} & \frac{11}{84} \quad 0
 \end{array} \tag{6.6}$$

The *FSAL* (First Same As Last) property means that stage k_7 of an accepted step is identical

to stage k_1 of the next step. This reduces the effective cost to **6 RHS evaluations per accepted step** rather than 7.

6.3.2 PI Stepsize Controller

LiQuID uses the Gustafsson (1991) PI controller for adaptive step size selection:

$$h_{n+1} = h_n \cdot S \cdot \left(\frac{1}{\varepsilon_n} \right)^{\alpha_{\text{PI}}} \cdot (\varepsilon_{n-1})^{\beta_{\text{PI}}} \quad (6.7)$$

where ε_n is the normalised local error estimate, $S = 0.9$ is the safety factor, $\alpha_{\text{PI}} = 0.7/5$, and $\beta_{\text{PI}} = 0.4/5$ (default values calibrated for the 5th-order DOPRI method). The factor is clipped to $[0.2, 10]$ to prevent runaway step changes.

6.3.3 FSAL Implementation

```

1  // In DormandPrince45::try_step() -- excerpt:
2
3  if (/* step accepted */) {
4      // Reuse k7 (the 7th stage of this step) as k1 of the next step.
5      // k7 = f(t + h, psi5) = f(t_new, psi_new)
6      // This is FSAL: saves one RHS evaluation per accepted step.
7      k1_.copy_from(k7_);
8      fsal_valid_ = true;
9      // ...
10 }
11
12 // In compute_stages(), on the first stage:
13 if (!fsal_valid_) {
14     k1_.set_zero();
15     rhs_fn(t, y0, k1_); // Evaluate only if FSAL not available
16     ++rhs_count_;
17 }
18 // Otherwise, k1_ already holds the correct value -- no RHS call.
```

FSAL reduces the typical DOPRI45 cost from 7 to 6 RHS evaluations per step (a 14% saving). The stepper's `reset()` method sets `fsal_valid_ = false`, ensuring that after a quantum jump (which resets the stepper), the first stage is recomputed correctly from the post-jump wavefunction.

6.3.4 Configuration

```

1  ode::DormandPrince45Config cfg;
2  cfg.atol      = 1e-8;    // absolute tolerance
3  cfg.rtol      = 1e-6;    // relative tolerance
4  cfg.dt_initial = 1e-3;
5  cfg.dt_min     = 1e-12;  // floor
6  cfg.dt_max     = 1e-1;   // ceiling
7  cfg.safety     = 0.9;    // PI controller safety factor
```

```
8   cfg.alpha      = 0.7/5.0; // PI integral gain
9   cfg.beta       = 0.4/5.0; // PI derivative gain
10  cfg.max_rejects = 100;    // max rejections before failure
11
12  ode::DormandPrince45 stepper(cfg);
```


Chapter 7

Layer 2: Quantum System

7.1 LindbladSet

`LindbladSet<Tag>` stores the collection of jump operators $\{L_k\}_{k=1}^K$ and their precomputed aggregate:

$$\Gamma = \sum_{k=1}^K L_k^\dagger L_k \quad (7.1)$$

Two specializations exist:

- `LindbladSet<DenseTag>` — uses `DenseOperator`.
- `LindbladSet<SparseTag>` — uses `SparseOperator`.

Both have an identical public interface.

7.1.1 Why Precompute Γ ?

The non-Hermitian Hamiltonian is $H_{\text{eff}} = H - \frac{i}{2}\Gamma$. At every ODE sub-step, the propagator must evaluate:

$$-iH_{\text{eff}} |\psi\rangle = -iH |\psi\rangle - \frac{1}{2}\Gamma |\psi\rangle \quad (7.2)$$

Without precomputation of Γ , this would require K separate matrix-vector products $L_k^\dagger L_k |\psi\rangle$ at every step — $\mathcal{O}(K \cdot N^2)$ work. Precomputing Γ reduces this to a single product $\Gamma |\psi\rangle$ — $\mathcal{O}(N^2)$, independent of K . For $K = 5$ operators, this is a $5\times$ reduction in the dominant inner-loop cost.

7.1.2 Construction and Immutability

```

1 // Dense specialization
2 std::vector<DenseOperator> ops;
3 ops.push_back(make_sigma_minus(gamma_1));
4 ops.push_back(make_sigma_minus(gamma_2)); // two decay channels
5
6 LindbladSet<DenseTag> L(std::move(ops));
7 // ~ Internally calls precompute_gamma():
8 //   gamma_ = L[0]dag*L[0] + L[1]dag*L[1]
9
10 // Copy is disabled (prevents accidental stale-gamma bugs):
11 // LindbladSet<DenseTag> L2 = L; // COMPILE ERROR
12
13 // Move is allowed:
14 LindbladSet<DenseTag> L3 = std::move(L); // OK

```

INV- Γ : $\Gamma \equiv \sum_k L_k^\dagger L_k$ is established at construction and is immutable thereafter. The copy constructor is deleted to prevent the creation of a `LindbladSet` with a stale Γ . Parameter sweeps must construct new `LindbladSet` instances for each parameter value.

7.1.3 Hot-Path Interface

```

1 // Called at every ODE sub-step.
2 // Computes: out += -(i/2) * Gamma * psi
3 // where Gamma = sum_k LkdagLk (precomputed)
4 void LindbladSet<DenseTag>::apply_decay(
5     const StateVector& psi,
6     StateVector& out) const noexcept;
7 // alpha = -(i/2) = Scalar{0.0, -0.5} is hard-coded

```

7.1.4 Cold-Path Interface

```

1 std::size_t K = L.num_channels(); // K = number of operators
2 Dim        N = L.hilbert_dim();  // Hilbert space dimension
3
4 // Evaluate L[k] * psi, store in out (used for jump probability)
5 StateVector scratch(N);
6 L.apply_channel(k, psi, scratch); // out = L[k] * psi
7
8 // Compute p[k] = ||L[k] * psi||^2 for all k
9 std::vector<Real> probs(K);
10 Real total = L.all_probabilities(psi, scratch, probs);
11 // probs[k] = ||L[k] * psi||^2, total = sum(probs)

```

7.2 OpenSystem

`OpenSystem<TimeTag, StorageTag>` encapsulates the complete physics description of the open quantum system.

7.2.1 Dense Time-Independent System

This is the primary type for Phase 1 simulations and is aliased as `DenseOpenSystem`:

```

1 // Construct from Hamiltonian and Lindblad operators.
2 // Preconditions:
3 //   H.size() == L.hilbert_dim()
4 //   Both must be constructed before being moved in.
5 DenseOpenSystem sys(std::move(H), std::move(L));
6 // ^ Immediately calls precompute_H_eff():
7 //   H_eff_ = H - (i/2) * Gamma
8
9 // HOT PATH: d|psi>/dt = -i * H_eff * psi
10 // Sets out = -i * H_eff * psi (out is overwritten, not accumulated)
11 sys.apply_Heff(psi, out);
12
13 // COLD PATH: jump operations
14 std::vector<Real> probs(sys.num_channels());
15 sys.jump_probabilities(psi, scratch, probs); // fills probs
16 sys.apply_jump(k, psi, scratch);             // psi <- Lk*psi/||Lk*psi||
17
18 // Introspection
19 sys.hilbert_dim(); // N
20 sys.num_channels(); // K
21 sys.H_eff();        // const ref to precomputed H_eff

```

7.2.2 Precomputed H_{eff}

```

1 // Excerpt from precompute_H_eff():
2 void precompute_H_eff() {
3     H_eff_ = H; // H_eff = H
4     constexpr Scalar neg_i_half{0.0, -0.5}; // -(i/2)
5     H_eff_.add_scaled(L_.decay_operator(), // += -(i/2) * Gamma
6                       neg_i_half);
7 }
8 // Result: H_eff = H - (i/2) * Gamma (frozen at construction)

```

The constant $-i/2 = \{0.0, -0.5\}$ appears in three places: `precompute_H_eff()`, `apply_decay()`, and the `apply_Heff()` call to `H_eff_.apply_add(psi, -i_unit, out)`.

7.2.3 Sparse Time-Independent System

```

1 // Construct with sparse operators
2 SparseOpenSystem sys_sparse(
3     SparseOperator H_sparse,
4     LindbladSet<SparseTag> L_sparse);

```

```

5
6 // Interface identical to DenseOpenSystem:
7 sys_sparse.apply_Heff(psi, out);
8 sys_sparse.apply_jump(k, psi, scratch);

```

7.2.4 Time-Dependent Hamiltonian

For $H(t) = H_{\text{static}} + \sum_j f_j(t) H_{\text{drive},j}$:

```

1 // DriveTerm: one time-dependent drive term
2 DriveTerm<DenseTag> drive;
3 drive.envelope = [](Real t) { return std::cos(omega_d * t); };
4 drive.H_drive = make_H_drive_dense(N, coupling_g);
5
6 // Construct time-dependent open system
7 auto sys_td = OpenSystem<TimeDependentTag, DenseTag>(
8     std::move(H_static),
9     std::move(L),
10    std::vector<DriveTerm<DenseTag>>{std::move(drive)});
11
12 // HOT PATH (now takes time argument):
13 // out = -i * [H_static + sum_j f_j(t)*H_drive_j - (i/2)*Gamma] * psi
14 sys_td.apply_Heff(t, psi, out);

```


Layer 3: Trajectory

8.1 TrajectoryState

TrajectoryState is the complete mutable record of a single MCWF trajectory. It is always created via the factory function `make_trajectory_state()` and modified exclusively by `MCWFPropagator::advance()`.

8.1.1 CoreState

```
1  struct CoreState {
2      StateVector    psi;           // current wavefunction [INV-1]
3      Real           t;             // current time [INV-4]
4      Real           t_final;       // end time -- immutable [INV-7]
5      Real           r;             // jump threshold in (0,1] [INV-2]
6      Real           dt_last;       // warm-start hint for next step
7      Real           err_last;      // last local error estimate
8      RNGState       rng;           // trajectory-exclusive RNG [INV-3]
9      TrajId         traj_id;       // immutable identifier [INV-6]
10     TrajectoryStatus status;       // lifecycle enum
11 };
```

The seven invariants of CoreState are defined in section 4.1 and enforced exclusively by `MCWFPropagator`.

8.1.2 Factory Function

```
1  // The ONLY legal way to create a TrajectoryState.
2  TrajectoryState ts = make_trajectory_state(
3      psi0,                // initial state (normalized)
4      0.0,                 // t0
5      10.0,                // t_final
6      traj_id,             // unique ID (e.g., 0, 1, 2, ...)
7      global_seed,         // seed for this trajectory's RNG
```

```

8     DiagnosticLevel::Summary           // optional, default Summary
9 );
10 // Postconditions:
11 //   ts.core.psi.norm_sq() ~= 1.0
12 //   ts.core.r is a fresh draw in (0,1]
13 //   ts.core.status == TrajectoryStatus::Initialized

```

8.1.3 DiagnosticRecord

```

1  if (ts.has_diagnostics()) {
2      const auto& diag = ts.diag();
3
4      std::printf("Total steps : %llu\n", diag.total_steps);
5      std::printf("Total jumps : %llu\n", diag.total_jumps);
6      std::printf("Mean dt      : %.3e\n", diag.mean_dt);
7      std::printf("Min dt       : %.3e\n", diag.min_dt);
8      std::printf("Wall time    : %.3f s\n", diag.wall_time_s);
9
10     // Jump events (requires DiagnosticLevel::Summary or higher)
11     for (const auto& jump : diag.jumps) {
12         std::printf("  Jump at t=%.4f channel=%zu norm_before=%.6f\n",
13             jump.t, jump.channel, jump.norm_before);
14     }
15 }

```

8.2 MCWFPropagator

MCWFPropagator<SystemType, StepperType> implements the MCWF algorithm. It is the computational engine of LiQuID.

8.2.1 Per-Instance Scratch Space

Five StateVector buffers are allocated at construction time and reused for every call to advance():

Table 8.1: MCWFPropagator per-instance scratch buffers.

Member	Size	Purpose
psi_scratch_	16 <i>N</i> bytes	General-purpose temporary
psi_proposed_	16 <i>N</i> bytes	ODE step output before jump check
psi_before_	16 <i>N</i> bytes	State saved before each ODE step
accum_tmp_	16 <i>N</i> bytes	RHS accumulation temporary
prob_scratch_	8 <i>K</i> bytes	Per-channel jump probabilities
Total	64 <i>N</i> + 8 <i>K</i> bytes	(e.g., ~ 4.1 kB for $N = 64, K = 3$)

MCWFPropagator is **not thread-safe**. Each thread must own its own propagator instance. Multiple propagators may safely share a single `const SystemType*` pointer because `OpenSystem` is immutable and `apply_Heff()` is declared `const noexcept`.

8.2.2 Direct Usage

Direct use of `MCWFPropagator` is appropriate when fine-grained control over a single trajectory is required — for example, in debugging, visualization, or single-trajectory benchmarking:

```

1  #include "liquid/liquid.hpp"
2  using namespace liquid;
3  using namespace liquid::ode;
4  using namespace liquid::trajectory;
5
6  int main() {
7      // Build system
8      DenseOperator H(2);
9      H(0,0) = Scalar{0.5, 0.0}; H(1,1) = Scalar{-0.5, 0.0};
10     DenseOperator L(2);
11     L(1,0) = Scalar{1.0, 0.0};
12
13     DenseOpenSystem sys(H, LindbladSet<DenseTag>({L}));
14
15     // Create trajectory
16     StateVector psi0(2);
17     psi0[0] = Scalar{1.0, 0.0};
18
19     auto ts = make_trajectory_state(psi0, 0.0, 5.0,
20                                   /*traj_id=*/0,
21                                   /*seed=*/42,
22                                   DiagnosticLevel::Full);
23
24     // Create propagator with RK4 stepper
25     PropagatorConfig cfg;
26     cfg.dt_initial = 1e-3;
27     MCWFPropagator<DenseOpenSystem, RK4Stepper> prop(
28         &sys,
29         RK4Stepper(cfg.dt_initial),
30         cfg);
31
32     // Event-driven loop: inspect state after every jump
33     using liquid::trajectory::StepOutcome;
34     StepOutcome outcome = StepOutcome::Advanced;
35
36     while (outcome != StepOutcome::Completed &&
37           outcome != StepOutcome::Failed) {
38         outcome = prop.advance(ts, ts.core.t_final);
39
40         if (outcome == StepOutcome::JumpOccurred) {
41             std::printf("Jump at t=%.4f ||psi||^2 = %.6f\n",
42                         ts.core.t,
43                         ts.core.psi.norm_sq());
44         }
45     }

```

```

46
47     if (ts.is_completed()) {
48         std::printf("Trajectory completed. Total jumps: %llu\n",
49                     ts.diag().total_jumps);
50     } else {
51         std::printf("Trajectory FAILED at t=%.4f\n", ts.core.t);
52     }
53
54     return 0;
55 }

```

8.2.3 Convenience: run_to_completion

```

1  // Equivalent to calling advance() in a loop until Completed or Failed.
2  // Records total wall time in ts.diag().wall_time_s.
3  TrajectoryStatus status = prop.run_to_completion(ts);
4
5  if (status == TrajectoryStatus::Completed) {
6      // Compute observable
7      DenseOperator sz(2);
8      sz(0,0) = Scalar{1,0}; sz(1,1) = Scalar{-1,0};
9      Scalar ev = expectation(sz, ts.core.psi);
10     std::printf("<sz> = %.4f\n", ev.real());
11 }

```

Layer 4: Ensemble Management

9.1 ObservableAccumulator

ObservableAccumulator bridges the physics layer (wavefunctions) and the statistics layer (Welford online estimators).

9.1.1 Observable Definitions

An observable is defined as a callable ObservableFn:

```
1  using ObservableFn = std::function<Real(const StateVector&);>;
2
3  // Example: expectation value of sigma_z
4  DenseOperator sz(2);
5  sz(0,0) = Scalar{1,0}; sz(1,1) = Scalar{-1,0};
6
7  // The ObservableFn must normalize internally:
8  // <O> = <psi|O|psi> / <psi|psi>
9  ObservableDef sz_def;
10 sz_def.name = "sigma_z";
11 sz_def.eval = [op = sz](const StateVector& psi) -> Real {
12     const Real nrm2 = psi.norm_sq();
13     if (nrm2 < 1e-14) return 0.0;
14     return expectation(op, psi).real() / nrm2;
15 };
```

The `SimulationBuilder::observe("name", DenseOperator)` convenience method automatically wraps the operator in a normalized expectation value callable. Use the raw `ObservableFn` interface only when computing custom quantities (e.g., photon number variance, Wigner function samples, or operator correlators).

9.2 ConvergenceMonitor

ConvergenceMonitor answers the question “should the ensemble stop?” It is stateless except for an atomic stop flag:

```

1  StoppingCriteria sc;
2  sc.target_rel_sem = 0.01;    // 1% relative SEM
3  sc.min_trajectories = 100;
4  sc.max_trajectories = 50000;
5
6  ConvergenceMonitor monitor(sc);
7
8  // After each trajectory batch:
9  ConvergenceReport report = monitor.evaluate(accum, elapsed_seconds);
10
11 switch (report.decision) {
12     case StoppingDecision::Continue:    /* keep going */ break;
13     case StoppingDecision::Converged:
14         std::printf("Converged at N=%zu, rel_sem=%.4f\n",
15                     report.trajectories_completed, report.current_rel_sem);
16         break;
17     case StoppingDecision::BudgetHit:
18         std::printf("Budget exhausted: %s\n",
19                     report.reason.c_str());
20         break;
21     case StoppingDecision::Forced:
22         std::printf("Stop requested externally.\n");
23         break;
24 }
```

9.3 Adaptive Allocator and Policies

The allocator policy is a `std::function<AllocationDecision(EnsembleSummary)>`. The default is `make_uniform_policy()`: always spawn a new trajectory.

```

1  // Uniform policy (default): always new trajectory, round-robin seeds
2  AllocatorPolicy uniform = make_uniform_policy(global_seed);
3
4  // Greedy policy: prioritise seeds that minimise current SEM
5  AllocatorPolicy greedy = make_greedy_policy(global_seed);
6
7  // ML policy (Phase 6): 5-feature MLP guiding seed selection
8  AllocatorPolicy ml = ensemble::ml::make_ml_policy(global_seed);
```

The `TrajectorySummary` struct (with fields `total_jumps`, `total_steps`, `wall_time_seconds`, `mean_dt`) is the primary feature vector for the ML policy. It is also the data format for logging and post-hoc analysis.

9.4 EnsembleManager

EnsembleManager<SystemType, StepperType> orchestrates multi-threaded trajectory dispatch via OpenMP:

```
1 // Typically not instantiated directly: use SimulationBuilder.
2 // Shown here for illustration of the threading model.
3 EnsembleManager<DenseOpenSystem, RK4Stepper> manager(
4     &sys, // shared, read-only
5     std::move(observable_defs),
6     stopping_criteria,
7     ensemble_config, // num_threads, seed, diag_level
8     make_uniform_policy(42));
9
10 EnsembleResult result = manager.run(psi0, 0.0, t_final);
11
12 std::printf("N=%zu failed=%zu jumps_mean=%.2f wall=%.2fs\n",
13     result.total_trajectories,
14     result.failed_trajectories,
15     result.mean_jumps_per_trajectory,
16     result.total_wall_time_seconds);
```


Layer 5: Simulation Interface

10.1 SimulationBuilder

`SimulationBuilder` is the primary user-facing entry point. It uses the fluent builder pattern to accumulate configuration and produce a type-erased `Simulation` handle via `build()`.

10.1.1 Complete Builder Reference

```

1  Simulation sim = SimulationBuilder{
2      // -- Physics -----
3      .hamiltonian(H)           // DenseOperator: system Hamiltonian
4      .collapse_operator(L1)    // Add first jump operator
5      .collapse_operator(L2)    // Chain multiple operators
6      // -- Observables -----
7      .observe("sz", sz)        // Dense operator observable
8      .observe("n_photon", [](const StateVector& psi) -> Real {
9          // Custom observable (e.g., photon number)
10         Real n = 0.0;
11         for (Idx i = 0; i < psi.size(); ++i)
12             n += static_cast<Real>(i) * std::norm(psi[i]);
13         return n / psi.norm_sq();
14     })
15     // -- ODE solver -----
16     .dt(1e-3)                  // PropagatorConfig::dt_initial
17     .solver(SolverType::DormandPrince45) // default: RK4Fixed
18     .atol(1e-8)                // DOPRI45: absolute tolerance
19     .rtol(1e-6)                // DOPRI45: relative tolerance
20     // -- Stopping -----
21     .stop_at_sem(0.01)         // StoppingCriteria::target_rel_sem
22     .min_trajectories(100)
23     .max_trajectories(10000)
24     .max_wall_time(60.0)       // seconds; 0 = no limit
25     // -- Parallelism -----
26     .num_threads(4)            // requires -fopenmp
27     .seed(42)

```

```

28 // -- Diagnostics -----
29 .diagnostic_level(DiagnosticLevel::Summary)
30 .build();

```

10.1.2 Running and Inspecting Results

```

1  auto result = sim.run(psi0, t0, t_final);
2
3  // Per-observable statistics
4  for (const auto& obs : result.observables) {
5      std::printf("%-20s mean=%+.6f sem=%.2e rel_sem=%.2e N=%zu\n",
6                  obs.name.c_str(),
7                  obs.mean,
8                  obs.sem,
9                  obs.rel_sem,
10                 result.total_trajectories);
11  }
12
13  // Ensemble metadata
14  std::printf("Convergence: %s\n", result.convergence.reason.c_str());
15  std::printf("Threads used: %zu\n", result.num_threads_used);
16  std::printf("Mean jumps/traj: %.2f\n", result.mean_jumps_per_trajectory);

```

10.2 Parameter Sweep

ParameterSweepBuilder runs the same simulation at multiple parameter values and collects results with full statistical metadata.

```

1  #include "liquid/simulation/parameter_sweep.hpp"
2  using namespace liquid;
3
4  int main() {
5      // Shared operators (not parameter-dependent)
6      DenseOperator sz(2);
7      sz(0,0) = Scalar{1,0}; sz(1,1) = Scalar{-1,0};
8
9      StateVector psi0(2);
10     psi0[0] = Scalar{1.0, 0.0}; // excited state
11
12     // Define the sweep
13     auto sweep = ParameterSweepBuilder{
14         .parameter("gamma", {0.1, 0.5, 1.0, 2.0, 5.0, 10.0})
15         .simulation_factory([&sz](double gamma) {
16             DenseOperator H(2);
17             H(0,0) = Scalar{0.5, 0.0}; H(1,1) = Scalar{-0.5, 0.0};
18
19             DenseOperator L(2);
20             L(1,0) = Scalar{std::sqrt(gamma), 0.0};
21

```

```

22         return SimulationBuilder{
23             .hamiltonian(std::move(H))
24             .collapse_operator(std::move(L))
25             .observe("sigma_z", sz)
26             .seed(42)
27             .dt(1e-3)
28             .stop_at_sem(0.01)
29             .min_trajectories(100)
30             .max_trajectories(5000)
31             .build();
32     })
33     .initial_state(psi0)
34     .time_interval(0.0, 5.0)
35     .build();
36
37     SweepResult result = sweep.run();
38
39     // Save to CSV: param,obs_name,mean,sem,rel_sem,N_trajectories
40     result.save_csv("sweep_gamma.csv");
41
42     // Access individual points
43     for (const auto& pt : result.points) {
44         const auto& obs = pt.result.observables[0];
45         std::printf("gamma=%.2f  <sz>=%.4f +/- %.4f  (N=%zu)\n",
46             pt.param_value, obs.mean, obs.sem,
47             pt.result.total_trajectories);
48     }
49
50     return 0;
51 }

```


Part III

Examples and Validation

Physics Examples

11.1 Example 1: Two-Level Spontaneous Decay

The paradigmatic open quantum system: a two-level atom at frequency ω , decaying at rate γ via spontaneous emission.

Exact solution:

$$\langle \sigma_z(t) \rangle = (P_e(0) - P_g(0)) e^{-\gamma t} + (P_g(0) - P_e(0)) \quad (11.1)$$

For initial excited state $|e\rangle$: $P_e(0) = 1$, so $\langle \sigma_z \rangle = 2e^{-\gamma t} - 1$.

```
1  #include "liquid/liquid.hpp"
2  #include <cmath>
3  #include <cstdio>
4  using namespace liquid;
5  using namespace liquid::ensemble;
6
7  int main() {
8      const double omega = 1.0;
9      const double gamma = 1.0;
10     const double T      = 5.0;
11
12     // Hamiltonian
13     DenseOperator H(2);
14     H(0,0) = Scalar{ omega/2, 0}; H(1,1) = Scalar{-omega/2, 0};
15
16     // Jump operator: sigma_minus = sqrt(gamma) |g><e|
17     DenseOperator L(2);
18     L(1,0) = Scalar{std::sqrt(gamma), 0};
19
20     // Observable: sigma_z = |e><e| - |g><g|
21     DenseOperator sz(2);
22     sz(0,0) = Scalar{1, 0}; sz(1,1) = Scalar{-1, 0};
23
24     // Build and run
25     Simulation sim = SimulationBuilder{}
```

```

26     .hamiltonian(std::move(H))
27     .collapse_operator(std::move(L))
28     .observe("sigma_z", std::move(sz))
29     .seed(1337)
30     .dt(1e-3)
31     .stop_at_sem(0.005)           // 0.5% SEM target
32     .min_trajectories(200)
33     .max_trajectories(20000)
34     .build();
35
36     StateVector psi0(2);
37     psi0[0] = Scalar{1.0, 0.0}; // excited state
38
39     auto result = sim.run(psi0, 0.0, T);
40
41     const double exact = 2.0 * std::exp(-gamma * T) - 1.0;
42     const double error = std::abs(result.observables[0].mean - exact);
43     const double nsig = error / result.observables[0].sem;
44
45     std::printf("=== Two-Level Spontaneous Decay ===\n");
46     std::printf("  <sigma_z> MCWF : %+.6f +/- %.6f\n",
47         result.observables[0].mean,
48         result.observables[0].sem);
49     std::printf("  <sigma_z> Exact: %+.6f\n", exact);
50     std::printf("  Deviation      : %.2f sigma\n", nsig);
51     std::printf("  Trajectories      : %zu\n", result.total_trajectories);
52     std::printf("  Mean jumps/traj   : %.2f\n",
53         result.mean_jumps_per_trajectory);
54
55     // Validation: result must be within 3 sigma of exact
56     return (nsig < 3.0) ? 0 : 1;
57 }

```

11.2 Example 2: Sparse Jaynes–Cummings Steady State

The Jaynes–Cummings model describes a two-level atom coupled to a single electromagnetic cavity mode. At steady state with cavity decay κ and atomic decay γ , the mean photon number $\langle a^\dagger a \rangle_{\text{ss}}$ can be computed analytically.

System Hamiltonian (rotating frame):

$$H = \Delta_c a^\dagger a + \Delta_a \sigma_+ \sigma_- + g(a \sigma_+ + a^\dagger \sigma_-) + \Omega(a + a^\dagger) \quad (11.2)$$

where g is the vacuum Rabi coupling, Ω the drive amplitude, and $\Delta_{c,a}$ the cavity and atom detunings.

11.3 Example 2: Driven Two-Level System

A coherent drive (Rabi frequency Ω) competes with spontaneous emission (rate γ). In the rotating frame at the resonance frequency, the Hamiltonian is $H = \Omega \sigma_x$ and the single jump operator is $L = \sqrt{\gamma} \sigma_-$. The optical Bloch equations give the steady-state excited population:

$$\varrho_{ee}^{\text{ss}} = \frac{4\Omega^2}{\gamma^2 + 8\Omega^2} \quad (11.3)$$

Note the factor of 4 (not 1): it arises because $H = \Omega \sigma_x$ (not $\frac{\Omega}{2} \sigma_x$). eq. (11.3) is verified numerically for two drive regimes.

```

1  #include "liquid/liquid.hpp"
2  #include <cmath>
3  #include <cstdio>
4  using namespace liquid;
5  using namespace liquid::ensemble;
6
7  // Analytic steady-state excited population for H = Omega*sigma_x,
8  // L = sqrt(gamma)*sigma_minus.
9  static double analytic_rho_ee_ss(double Omega, double gamma) {
10     const double r = Omega / gamma;
11     return 4.0 * r * r / (1.0 + 8.0 * r * r);
12 }
13
14 // Build the rotating-frame driven open system
15 static DenseOpenSystem make_driven_system(double Omega, double gamma) {
16     // H = Omega * sigma_x (resonant drive, rotating frame)
17     DenseOperator H(2);
18     H(0, 1) = Scalar{Omega, 0.0};
19     H(1, 0) = Scalar{Omega, 0.0};
20
21     // L = sqrt(gamma) * sigma_minus
22     DenseOperator L(2);
23     L(1, 0) = Scalar{std::sqrt(gamma), 0.0};
24
25     std::vector<DenseOperator> ops;
26     ops.push_back(std::move(L));
27     return DenseOpenSystem(std::move(H),
28         ↳ LindbladSet<DenseTag>(std::move(ops)));
29 }
30
31 int main() {
32     const double gamma = 1.0;
33     const double T_ss = 20.0; // long enough to reach steady state
34     const int N_traj = 3000;
35
36     // -- Regime 1: weak drive (Omega = 0.1*gamma) -----
37     {
38         const double Omega = 0.1 * gamma;
39         const double exact_ss = analytic_rho_ee_ss(Omega, gamma);
40
41         auto sys = make_driven_system(Omega, gamma);
42
43         PropagatorConfig cfg;

```

```

43     cfg.dt_initial = 5e-4; // small dt: Rabi period = pi/(Omega) ~ 31
44
45     double sum_rho = 0.0, sum_rho2 = 0.0;
46     StateVector psi0(2);
47     psi0[0] = Scalar{1.0, 0.0}; // start excited
48
49     for (int traj = 0; traj < N_traj; ++traj) {
50         liquid::ode::RK4Stepper stepper(cfg.dt_initial);
51         liquid::trajectory::MCWFPropagator<
52             DenseOpenSystem, liquid::ode::RK4Stepper> prop(&sys, stepper,
53                 ↪ cfg);
54
55         auto ts = liquid::trajectory::make_trajectory_state(
56             psi0, 0.0, T_ss,
57             static_cast<TrajId>(traj), 54321ULL,
58             DiagnosticLevel::None);
59
60         prop.run_to_completion(ts);
61
62         const Real ns      = ts.core.psi.norm_sq();
63         const double rho_e = ns > 1e-30
64             ? std::norm(ts.core.psi[0]) / ns
65             : 0.0;
66         sum_rho += rho_e;
67         sum_rho2 += rho_e * rho_e;
68     }
69
70     const double mean = sum_rho / N_traj;
71     const double var  = sum_rho2 / N_traj - mean * mean;
72     const double sem  = std::sqrt(std::max(0.0, var) / N_traj);
73
74     std::printf("Weak drive (Omega/gamma = %.2f):\n", Omega / gamma);
75     std::printf("  rho_ee(MCWF) = %.4f +/- %.4f\n", mean, sem);
76     std::printf("  rho_ee(exact) = %.4f\n", exact_ss);
77     std::printf("  |error|      = %.4f  (%.1f sigma)\n\n",
78         std::abs(mean - exact_ss),
79         std::abs(mean - exact_ss) / sem);
80 }
81
82 // -- Regime 2: strong drive (Omega = gamma) -----
83 {
84     const double Omega      = 1.0 * gamma;
85     const double exact_ss   = analytic_rho_ee_ss(Omega, gamma);
86
87     auto sys = make_driven_system(Omega, gamma);
88
89     PropagatorConfig cfg;
90     cfg.dt_initial = 5e-4;
91
92     double sum_rho = 0.0, sum_rho2 = 0.0;
93     StateVector psi0(2);
94     psi0[0] = Scalar{1.0, 0.0};
95
96     for (int traj = 0; traj < N_traj; ++traj) {
97         liquid::ode::RK4Stepper stepper(cfg.dt_initial);
98         liquid::trajectory::MCWFPropagator<

```

```

98         DenseOpenSystem, liquid::ode::RK4Stepper> prop(&sys, stepper,
99         ↪   cfg);
100
101     auto ts = liquid::trajectory::make_trajectory_state(
102         psi0, 0.0, T_ss,
103         static_cast<TrajId>(traj), 99999ULL,
104         DiagnosticLevel::None);
105
106     prop.run_to_completion(ts);
107
108     const Real ns      = ts.core.psi.norm_sq();
109     const double rho_e = ns > 1e-30
110         ? std::norm(ts.core.psi[0]) / ns
111         : 0.0;
112     sum_rho += rho_e;
113     sum_rho2 += rho_e * rho_e;
114 }
115
116 const double mean = sum_rho / N_traj;
117 const double var  = sum_rho2 / N_traj - mean * mean;
118 const double sem  = std::sqrt(std::max(0.0, var) / N_traj);
119
120 std::printf("Strong drive (Omega/gamma = %.2f):\n", Omega / gamma);
121 std::printf("  rho_ee(MCWF) = %.4f +/- %.4f\n", mean, sem);
122 std::printf("  rho_ee(exact) = %.4f [saturation: 1/3 limit]\n",
123 ↪   exact_ss);
124 std::printf("  |error|      = %.4f (%.1f sigma)\n",
125     std::abs(mean - exact_ss),
126     std::abs(mean - exact_ss) / sem);
127
128 }
129
130 return 0;
131 }

```

Rabi oscillations limit. Setting $\gamma \rightarrow 0$ with $\Omega > 0$ yields nearly closed Rabi oscillations: $\langle \sigma_z(t) \rangle = \cos(2\Omega t) e^{-\gamma t/2}$. LindbladSet requires at least one operator, so use a very small $\gamma \approx 10^{-2}$ to approximate the closed-system limit numerically.

11.4 Example 3: Cavity QED — Jaynes–Cummings Model

The Jaynes–Cummings (JC) model describes a single two-level atom coupled to one cavity mode. The composite Hilbert space is $\mathcal{H} = \mathcal{H}_{\text{cav}} \otimes \mathcal{H}_{\text{atom}}$ with dimension $2N_{\text{Fock}}$.

Basis convention. A basis state $|n, s\rangle$ is indexed as $i = 2n + s$, where $n \in \{0, \dots, N_F - 1\}$ is the photon number and $s \in \{0 = e, 1 = g\}$ is the atomic spin.

Hamiltonian (on resonance, $\omega_c = \omega_a = \omega$):

$$H = \omega a^\dagger a + \frac{\omega}{2} \sigma_z + g(a^\dagger \sigma_- + a \sigma_+) \quad (11.4)$$

Conservation law. The operator $\hat{N} = a^\dagger a + \sigma_+ \sigma_-$ is conserved by H . Starting from $|1, e\rangle$ ($\hat{N} = 2$), the system oscillates within the two-state manifold $\{|1, e\rangle, |2, g\rangle\}$ — not between $|1, e\rangle$ and $|0, g\rangle$. This is an important subtlety verified by the test below.

```

1  #include "liquid/liquid.hpp"
2  #include <cmath>
3  #include <cstdio>
4  #include <vector>
5  using namespace liquid;
6  using namespace liquid::trajectory;
7  using namespace liquid::ode;
8
9  // Basis: |n, s> -> index 2*n + s (s=0: excited, s=1: ground)
10 static Idx jc_idx(int n, int s) { return static_cast<Idx>(2 * n + s); }
11
12 // Cavity photon loss: L1 = sqrt(kappa) * a
13 // a|n, s> = sqrt(n)|n-1, s>
14 static SparseOperator make_L_cavity(int N_fock, double kappa) {
15     const Dim dim = 2 * N_fock;
16     std::vector<Triplet> trips;
17     for (int n = 1; n < N_fock; ++n) {
18         const double v = std::sqrt(kappa * static_cast<double>(n));
19         trips.push_back({jc_idx(n-1, 0), jc_idx(n, 0), Scalar{v, 0.0}});
20         trips.push_back({jc_idx(n-1, 1), jc_idx(n, 1), Scalar{v, 0.0}});
21     }
22     return SparseOperator(dim, std::move(trips));
23 }
24
25 // Atomic decay: L2 = sqrt(gamma_a) * sigma_minus
26 // sigma_minus|n, e> = |n, g>
27 static SparseOperator make_L_atomic(int N_fock, double gamma_a) {
28     const Dim dim = 2 * N_fock;
29     std::vector<Triplet> trips;
30     const double v = std::sqrt(gamma_a);
31     for (int n = 0; n < N_fock; ++n)
32         trips.push_back({jc_idx(n, 1), jc_idx(n, 0), Scalar{v, 0.0}});
33     return SparseOperator(dim, std::move(trips));
34 }
35
36 // JC Hamiltonian: omega*adaga + (omega/2)*sigma_z + g*(adagsigma_- +
37 //   ↪ a*sigma_+)
38 // Coupling term: adagsigma_-|n, e> = sqrt(n+1)|n+1, g>
39 static SparseOperator make_H_jc(int N_fock, double omega, double g) {
40     const Dim dim = 2 * N_fock;
41     std::vector<Triplet> trips;
42
43     // Free cavity: omega * adaga diagonal
44     for (int n = 0; n < N_fock; ++n) {
45         const double diag = omega * static_cast<double>(n);
46         trips.push_back({jc_idx(n, 0), jc_idx(n, 0), Scalar{diag, 0.0}});
47         trips.push_back({jc_idx(n, 1), jc_idx(n, 1), Scalar{diag, 0.0}});
48     }
49
50     // Free atom: (omega/2)*sigma_z diagonal
51     for (int n = 0; n < N_fock; ++n) {
52         trips.push_back({jc_idx(n, 0), jc_idx(n, 0), Scalar{ omega/2.0, 0.0}});
53     }
54 }

```

```

52     trips.push_back({jc_idx(n,1), jc_idx(n,1), Scalar{-omega/2.0, 0.0}});
53 }
54
55 // JC interaction: adagsigma_ couples |n,e> -> |n+1,g>
56 for (int n = 0; n < N_fock - 1; ++n) {
57     const double v = g * std::sqrt(static_cast<double>(n + 1));
58     trips.push_back({jc_idx(n+1, 1), jc_idx(n, 0), Scalar{v, 0.0}}); //
59     ↪ adagsigma_-
60     trips.push_back({jc_idx(n, 0), jc_idx(n+1, 1), Scalar{v, 0.0}}); //
61     ↪ h.c. a*sigma_+
62 }
63 return SparseOperator(dim, std::move(trips));
64 }
65
66 // Photon number observable: n_hat = sum_n n * (|n,e><n,e| + |n,g><n,g|)
67 static ObservableDef make_photon_number_obs(int N_fock) {
68     const Dim dim = 2 * N_fock;
69     std::vector<Triplet> trips;
70     for (int n = 0; n < N_fock; ++n) {
71         const double val = static_cast<double>(n);
72         trips.push_back({jc_idx(n,0), jc_idx(n,0), Scalar{val, 0.0}});
73         trips.push_back({jc_idx(n,1), jc_idx(n,1), Scalar{val, 0.0}});
74     }
75     SparseOperator n_op(dim, std::move(trips));
76     return {"photon_number",
77         [op = std::move(n_op)](const StateVector& psi) -> Real {
78             const Real ns = psi.norm_sq();
79             return ns > 1e-30
80                 ? sparse_expectation(op, psi).real() / ns
81                 : 0.0;
82         }};
83 }
84
85 // Excited-state population observable
86 static ObservableDef make_excited_pop_obs(int N_fock) {
87     const Dim dim = 2 * N_fock;
88     std::vector<Triplet> trips;
89     for (int n = 0; n < N_fock; ++n)
90         trips.push_back({jc_idx(n,0), jc_idx(n,0), Scalar{1.0, 0.0}});
91     SparseOperator pe_op(dim, std::move(trips));
92     return {"excited_pop",
93         [op = std::move(pe_op)](const StateVector& psi) -> Real {
94             const Real ns = psi.norm_sq();
95             return ns > 1e-30
96                 ? sparse_expectation(op, psi).real() / ns
97                 : 0.0;
98         }};
99 }
100
101 int main() {
102     const int N_fock = 12; // Fock space truncation
103     const double omega = 1.0; // cavity / atom frequency
104     const double g = 0.5; // vacuum Rabi coupling
105     const double kappa = 0.2; // cavity photon loss rate
106     const double gamma_a = 0.05; // atomic decay rate
107     const double T = 5.0; // simulation time

```

```

106
107 // Build sparse JC system
108 SparseOperator H = make_H_jc(N_fock, omega, g);
109 SparseOperator L1 = make_L_cavity(N_fock, kappa);
110 SparseOperator L2 = make_L_atomic(N_fock, gamma_a);
111
112 std::vector<SparseOperator> ops;
113 ops.push_back(std::move(L1));
114 ops.push_back(std::move(L2));
115
116 SparseOpenSystem sys(
117     std::move(H),
118     LindbladSet<SparseTag>(std::move(ops)));
119
120 // Observables
121 std::vector<ObservableDef> obs_defs;
122 obs_defs.push_back(make_photon_number_obs(N_fock));
123 obs_defs.push_back(make_excited_pop_obs(N_fock));
124
125 // Stopping criteria
126 StoppingCriteria sc;
127 sc.min_trajectories = 500;
128 sc.max_trajectories = 5000;
129 sc.target_rel_sem = 0.03;
130
131 EnsembleConfig ec;
132 ec.global_seed = 42;
133 ec.diag_level = DiagnosticLevel::None;
134 ec.propagator.dt_initial = 5e-3;
135
136 // Use the sparse+DOPRI45 factory (SparseOpenSystem requires
137 // Simulation::make_sparse_dopri45; SimulationBuilder supports
138 // DenseOperator only in v0.7.0)
139 Simulation sim = Simulation::make_sparse_dopri45(
140     std::move(sys), std::move(obs_defs), sc, ec);
141
142 // Initial state: |n=1, e> -- one photon, atom excited
143 StateVector psi0(2 * N_fock);
144 psi0[jc_idx(1, 0)] = Scalar{1.0, 0.0};
145
146 auto result = sim.run(psi0, 0.0, T);
147
148 std::printf("=== Jaynes-Cummings (N_fock=%d, g/kappa=%.1f) ===\n",
149     N_fock, g / kappa);
150 std::printf(" <n_photon> = %.4f +/- %.4f (N=%zu)\n",
151     result.observables[0].mean, result.observables[0].sem,
152     result.total_trajectories);
153 std::printf(" <rho_e> = %.4f +/- %.4f\n",
154     result.observables[1].mean, result.observables[1].sem);
155 std::printf(" Mean jumps = %.2f per trajectory\n",
156     result.mean_jumps_per_trajectory);
157 std::printf(" Convergence = %s\n",
158     result.convergence.reason.c_str());
159
160 return 0;
161 }

```

The driven-dissipative Jaynes–Cummings parameter sweep (sweeping g/κ to observe photon blockade) is provided in full in `examples/jc_steady_state.cpp`, using `ParameterSweepBuilder::parameter_logrange()`.

11.5 Example 4: Parameter Sweeps

`ParameterSweepBuilder` runs the same simulation at a list of parameter values and collects fully converged `EnsembleResult` objects at each point. Two sweep modes are supported: linear (`.parameter("name", {v1, v2, ...})`) and log-spaced (`.parameter_logrange("name", lo, hi, N)`).

The example below sweeps $\Omega/\gamma \in \{0.1, 0.5, 1.0, 2.0\}$ for the driven two-level system and verifies the steady-state formula eq. (11.3) at each point.

```

1  #include "liquid/liquid.hpp"
2  #include <cmath>
3  #include <cstdio>
4  using namespace liquid;
5  using namespace liquid::ensemble;
6
7  int main() {
8      const double gamma = 1.0;
9      const double T_ss = 15.0; // sufficient time to reach steady state
10
11      // Initial state: excited |e>
12      StateVector psi0(2);
13      psi0[0] = Scalar{1.0, 0.0};
14
15      // Analytic steady-state formula
16      auto exact_ss = [&](double Omega) {
17          const double r = Omega / gamma;
18          return 4.0 * r * r / (1.0 + 8.0 * r * r);
19      };
20
21      // Define the sweep over Rabi frequency Omega
22      auto sweep = ParameterSweepBuilder{}
23          .parameter("Omega", {0.1, 0.5, 1.0, 2.0})
24          .simulation_factory([gamma](double Omega) {
25              // H = Omega * sigma_x (resonant drive)
26              DenseOperator H(2);
27              H(0, 1) = Scalar{Omega, 0.0};
28              H(1, 0) = Scalar{Omega, 0.0};
29
30              // L = sqrt(gamma) * sigma_minus
31              DenseOperator L(2);
32              L(1, 0) = Scalar{std::sqrt(gamma), 0.0};
33
34              // Observable: rho_ee = |e><e|
35              DenseOperator rho_ee_op(2);
36              rho_ee_op(0, 0) = Scalar{1.0, 0.0};

```

```

37
38         return SimulationBuilder{
39             .hamiltonian(std::move(H))
40             .collapse_operator(std::move(L))
41             .observe("rho_ee", std::move(rho_ee_op))
42             .seed(42)
43             .dt(5e-4)
44             .stop_at_sem(0.02)           // 2% relative SEM at each point
45             .min_trajectories(500)
46             .max_trajectories(5000)
47             .build();
48     })
49     .initial_state(psi0)
50     .time_interval(0.0, T_ss)
51     .build();
52
53     SweepResult result = sweep.run();
54
55     // Print table with comparison to analytic formula
56     std::printf("%-10s  %-12s  %-12s  %-10s  %-8s  %s\n",
57         "Omega", "rho_ee(MCWF)", "rho_ee(exact)", "rel_err(%)", "N",
58         ↪ "converged");
59     std::printf("%s\n", std::string(72, '-').c_str());
60
61     for (const auto& pt : result.points) {
62         const auto& obs = pt.result.observables[0];
63         const double exact = exact_ss(pt.param_value);
64         const double rel_err = 100.0 * std::abs(obs.mean - exact) / exact;
65
66         std::printf("%-10.4f  %-12.4f  %-12.4f  %-10.2f  %-8zu  %s\n",
67             pt.param_value,
68             obs.mean,
69             exact,
70             rel_err,
71             pt.result.total_trajectories,
72             pt.result.convergence.reason.c_str());
73     }
74
75     // Save full results (CSV columns: param,obs,mean,sem,rel_sem,N,...)
76     result.save_csv("sweep_Omega.csv");
77     std::printf("\nSaved: sweep_Omega.csv\n");
78
79     return 0;

```

When running sweeps with `num_threads > 1`, the threads are used *within* each simulation point (ensemble parallelism), not across sweep points. Sweep points are executed sequentially; this allows early stopping to adapt N independently at each point rather than committing to a fixed N before the sweep starts.

11.6 Example 5: Convergence-Based Stopping

StoppingCriteria provides three independent stopping conditions: SEM target, trajectory budget, and wall-clock budget. The ConvergenceMonitor checks them after every batch of completed trajectories. The example below demonstrates all three and verifies that the SEM scales as $1/\sqrt{N}$.

```

1  #include "liquid/liquid.hpp"
2  #include <cmath>
3  #include <cstdio>
4  using namespace liquid;
5  using namespace liquid::ensemble;
6
7  static DenseOperator make_H(double omega) {
8      DenseOperator H(2);
9      H(0,0) = Scalar{ omega/2.0, 0.0};
10     H(1,1) = Scalar{-omega/2.0, 0.0};
11     return H;
12 }
13 static DenseOperator make_L(double gamma) {
14     DenseOperator L(2);
15     L(1,0) = Scalar{std::sqrt(gamma), 0.0};
16     return L;
17 }
18 static DenseOperator make_sz() {
19     DenseOperator sz(2);
20     sz(0,0) = Scalar{1.0, 0.0};
21     sz(1,1) = Scalar{-1.0, 0.0};
22     return sz;
23 }
24
25 int main() {
26     const double omega = 1.0;
27     const double gamma = 1.0;
28     const double T      = 1.0;
29
30     StateVector psi0(2);
31     psi0[0] = Scalar{1.0, 0.0};
32
33     const double exact = 2.0 * std::exp(-gamma * T) - 1.0;
34
35     // -- 1. SEM-based stopping (1% relative SEM target) -----
36     {
37         Simulation sim = SimulationBuilder{
38             .hamiltonian(make_H(omega))
39             .collapse_operator(make_L(gamma))
40             .observe("sz", make_sz())
41             .seed(42)
42             .dt(1e-3)
43             .stop_at_sem(0.01)
44             .min_trajectories(50)
45             .max_trajectories(50000)
46             .build();
47
48         auto result = sim.run(psi0, 0.0, T);
49         const auto& obs = result.observables[0];

```

```

50
51     std::printf("=== SEM-based stopping ===\n");
52     std::printf("  <sz> = %.4f +/- %.4f  rel_sem = %.4f  N = %zu\n",
53               obs.mean, obs.sem, obs.rel_sem, result.total_trajectories);
54     std::printf("  Exact = %.4f  |error| = %.4f sigma\n",
55               exact, std::abs(obs.mean - exact) / obs.sem);
56     std::printf("  Decision: %s\n\n",
57               result.convergence.reason.c_str());
58 }
59
60 // -- 2. Wall-clock budget (0.3 s limit, impossibly tight SEM) -----
61 {
62     Simulation sim = SimulationBuilder{
63         .hamiltonian(make_H(omega))
64         .collapse_operator(make_L(gamma))
65         .observe("sz", make_sz())
66         .seed(999)
67         .dt(1e-3)
68         .stop_at_sem(1e-9)           // impossible: triggers budget
69         .min_trajectories(10)
70         .max_trajectories(1000000)
71         .max_wall_time(0.3)         // 300 ms hard limit
72         .build();
73
74     auto result = sim.run(psi0, 0.0, T);
75
76     std::printf("=== Wall-clock budget ===\n");
77     std::printf("  Decision : %s\n",
78               result.convergence.reason.c_str());
79     std::printf("  N achieved: %zu  wall = %.3f s\n\n",
80               result.total_trajectories,
81               result.total_wall_time_seconds);
82 }
83
84 // -- 3. SEM scales as 1/sqrt(N): verify with three ensemble sizes -----
85 {
86     const std::size_t sizes[3] = {100, 400, 1600};
87     double sems[3];
88
89     std::printf("=== SEM scaling: expected SEM ~ 1/sqrt(N) ===\n");
90     for (int k = 0; k < 3; ++k) {
91         Simulation sim = SimulationBuilder{
92             .hamiltonian(make_H(omega))
93             .collapse_operator(make_L(gamma))
94             .observe("sz", make_sz())
95             .seed(static_cast<Seed>(k * 777 + 42))
96             .dt(1e-3)
97             .min_trajectories(sizes[k])
98             .max_trajectories(sizes[k])
99             .build();
100
101         auto result = sim.run(psi0, 0.0, T);
102         sems[k] = result.observables[0].sem;
103         std::printf("  N = %4zu  SEM = %.5f\n", sizes[k], sems[k]);
104     }
105

```

```

106         // Ratios should be ~2 (since N quadruples each time)
107         std::printf(" SEM ratio N=100/400 = %.3f (expected ~2.0)\n",
108             sems[0] / sems[1]);
109         std::printf(" SEM ratio N=400/1600 = %.3f (expected ~2.0)\n",
110             sems[1] / sems[2]);
111     }
112
113     return 0;
114 }

```

`max_wall_time` measures elapsed real time from the first `sim.run()` call. It does *not* account for operator construction or build time. On heavily loaded machines, the wall clock continues to tick even while the process is descheduled. Use `max_trajectories` as the primary safety ceiling; `max_wall_time` is a soft backstop for interactive sessions.

11.7 Example 6: Sparse Hilbert Spaces

This example illustrates the three design patterns for sparse Hilbert-space simulations: (i) using `SparseOperator` directly for a high-dimensional bosonic mode, (ii) exploiting the `Simulation::make_sparse_dopri45` factory for non-builder-compatible system types, and (iii) verifying the sparse-dense equivalence for a small test system.

Pattern A: Bosonic Fock space (pure sparse)

For a single driven, damped harmonic oscillator (cavity without atom):

$$H = \Delta a^\dagger a + \varepsilon(a + a^\dagger), \quad L = \sqrt{\kappa} a \quad (11.5)$$

The steady-state mean photon number has the exact value $\langle a^\dagger a \rangle_{\text{ss}} = \varepsilon^2 / \kappa^2$ in the linear regime ($\varepsilon \ll \kappa$).

```

1  #include "liquid/liquid.hpp"
2  #include <cmath>
3  #include <cstdio>
4  #include <vector>
5  using namespace liquid;
6  using namespace liquid::ensemble;
7
8  // Build the annihilation operator a for an N_fock-dimensional Fock space.
9  // a|n> = sqrt(n)|n-1> for n >= 1; a|0> = 0
10 static SparseOperator make_a_fock(Dim N_fock) {
11     std::vector<Triplet> trips;
12     trips.reserve(N_fock - 1);
13     for (Dim n = 1; n < N_fock; ++n) {
14         trips.push_back({
15             n - 1, // row (output state index)
16             n,      // col (input state index)

```

```

17         Scalar{std::sqrt(static_cast<double>(n)), 0.0});
18     }
19     return SparseOperator(N_fock, std::move(trips));
20 }
21
22 // Driven harmonic oscillator Hamiltonian:
23 //  $H = \Delta * a + \epsilon * (a + a^\dagger)$ 
24 static SparseOperator make_H_harmonic(Dim N_fock, double Delta, double eps) {
25     std::vector<Triplet> trips;
26     // Diagonal:  $\Delta * n$ 
27     for (Dim n = 0; n < N_fock; ++n)
28         trips.push_back({n, n, Scalar{Delta * static_cast<double>(n), 0.0}});
29     // Drive:  $\epsilon * a$  (lower triangular)
30     for (Dim n = 1; n < N_fock; ++n)
31         trips.push_back({n-1, n, Scalar{eps *
32             ↪ std::sqrt(static_cast<double>(n)), 0.0}});
33     // Drive:  $\epsilon * a^\dagger$  (upper triangular)
34     for (Dim n = 1; n < N_fock; ++n)
35         trips.push_back({n, n-1, Scalar{eps *
36             ↪ std::sqrt(static_cast<double>(n)), 0.0}});
37     return SparseOperator(N_fock, std::move(trips));
38 }
39
40 // Cavity decay:  $L = \sqrt{\kappa} * a$ 
41 static SparseOperator make_L_harmonic(Dim N_fock, double kappa) {
42     std::vector<Triplet> trips;
43     for (Dim n = 1; n < N_fock; ++n)
44         trips.push_back({
45             n - 1, n,
46             Scalar{std::sqrt(kappa * static_cast<double>(n)), 0.0}});
47     return SparseOperator(N_fock, std::move(trips));
48 }
49
50 // Photon number observable:  $a^\dagger a = \text{diag}(0, 1, 2, \dots, N_{\text{fock}}-1)$ 
51 static ObservableDef make_n_obs(Dim N_fock) {
52     std::vector<Triplet> trips;
53     for (Dim n = 0; n < N_fock; ++n)
54         trips.push_back({n, n, Scalar{static_cast<double>(n), 0.0}});
55     SparseOperator n_op(N_fock, std::move(trips));
56     return {"photon_n",
57         [op = std::move(n_op)](const StateVector& psi) -> Real {
58             const Real ns = psi.norm_sq();
59             return ns > 1e-30
60                 ? sparse_expectation(op, psi).real() / ns
61                 : 0.0;
62         }};
63 }
64
65 int main() {
66     const Dim N_fock = 40; // truncation: must satisfy  $N \gg \langle n \rangle_{ss}$ 
67     const double Delta = 0.0; // on resonance with drive
68     const double kappa = 1.0; // cavity decay rate
69     const double eps = 0.2; // drive amplitude (weak:  $\epsilon \ll \kappa$ )
70     const double T_ss = 15.0; // run until steady state
71
72     // Exact steady-state photon number (linear cavity)

```

```

71     const double exact_n_ss = (eps * eps) / (kappa * kappa);
72
73     std::printf("N_fock=%zu Delta=%.1f kappa=%.1f eps=%.2f\n",
74               N_fock, Delta, kappa, eps);
75     std::printf("Exact <n>_ss = eps^2/kappa^2 = %.4f\n\n", exact_n_ss);
76
77     // Verify truncation is adequate: <n>_ss << N_fock
78     if (exact_n_ss > static_cast<double>(N_fock) / 2.0) {
79         std::fprintf(stderr,
80                     "WARNING: N_fock=%zu may be too small for <n>_ss=%.1f\n",
81                     N_fock, exact_n_ss);
82     }
83
84     // Build sparse system
85     SparseOperator H = make_H_harmonic(N_fock, Delta, eps);
86     SparseOperator L = make_L_harmonic(N_fock, kappa);
87
88     std::vector<SparseOperator> ops;
89     ops.push_back(std::move(L));
90     SparseOpenSystem sys(std::move(H), LindbladSet<SparseTag>(std::move(ops)));
91
92     // Report sparsity advantage over dense
93     const double nnz = static_cast<double>(sys.H_eff().nnz());
94     const double dense_sq = static_cast<double>(N_fock * N_fock);
95     std::printf("H_eff sparsity: %.2f%% (%g non-zeros vs %g dense)\n\n",
96               100.0 * nnz / dense_sq, nnz, dense_sq);
97
98     std::vector<ObservableDef> obs_defs;
99     obs_defs.push_back(make_n_obs(N_fock));
100
101     StoppingCriteria sc;
102     sc.min_trajectories = 300;
103     sc.max_trajectories = 5000;
104     sc.target_rel_sem = 0.02;
105
106     EnsembleConfig ec;
107     ec.global_seed = 42;
108     ec.diag_level = DiagnosticLevel::None;
109     ec.propagator.dt_initial = 5e-3;
110
111     // Factory for SparseOpenSystem + DOPRI45
112     Simulation sim = Simulation::make_sparse_dopri45(
113         std::move(sys), std::move(obs_defs), sc, ec);
114
115     // Initial state: vacuum |0>
116     StateVector psi0(N_fock);
117     psi0[0] = Scalar{1.0, 0.0};
118
119     auto result = sim.run(psi0, 0.0, T_ss);
120
121     const auto& obs = result.observables[0];
122     std::printf("=== Driven harmonic oscillator (N_fock=%zu) ===\n", N_fock);
123     std::printf("  <n>(MCWF) = %.4f +/- %.4f (N=%zu)\n",
124               obs.mean, obs.sem, result.total_trajectories);
125     std::printf("  <n>(exact) = %.4f\n", exact_n_ss);
126     std::printf("  |error| = %.4f (%.1f sigma)\n",

```

```
127         std::abs(obs.mean - exact_n_ss),  
128         std::abs(obs.mean - exact_n_ss) / obs.sem);  
129  
130     return 0;  
131 }
```

The Hamiltonian eq. (11.5) has $3N_F - 2$ non-zero elements (one diagonal plus two off-diagonals of length $N_F - 1$ each). For $N_F = 40$, this is 118 non-zeros versus 1600 for a dense matrix — a $13.6\times$ memory saving and a comparable reduction in matrix-vector product cost. The savings grow linearly with N_F : for $N_F = 200$, the ratio exceeds $66\times$.

Part IV

Performance and Extension

Performance Guide

12.1 Computational Complexity Model

Table 12.1: Computational cost of each layer per trajectory. N = Hilbert dimension, K = jump operators, M = ODE steps, J = number of jumps.

Operation	Cost	Notes
Dense <code>apply_add</code>	$\mathcal{O}(N^2)$	Dominates for $N > 32$
Sparse <code>apply_add</code>	$\mathcal{O}(\text{nnz})$	$\ll N^2$ for sparse systems
RK4 step	$4 \cdot \mathcal{O}(N^2)$	4 RHS evaluations
DOPRI45 step (accepted)	$6 \cdot \mathcal{O}(N^2)$	6 RHS (FSAL saves one)
Jump detection	$\mathcal{O}(1)$	Norm comparison only
Jump channel selection	$\mathcal{O}(KN^2)$	K matvecs to compute p_k
State renormalization	$\mathcal{O}(N)$	Linear sweep
Observable evaluation	$\mathcal{O}(N^2)$	Once per trajectory
Per-trajectory total	$\mathcal{O}(M \cdot N^2)$	Dominated by ODE steps
Ensemble total	$\mathcal{O}(M \cdot N^2 \cdot T_{\text{traj}})$	Linear in T

12.2 Memory Hierarchy Considerations

For $N \leq 64$, the wavefunction $|\psi\rangle$ fits in $64 \times 16 = 1$ kB, which resides in L1 cache. The RK4 scratch buffers (5 vectors = 5 kB) fit in L1/L2. A 64×64 dense matrix (= 64 kB) fits in L2. Keeping all working data in L1/L2 is the primary performance objective for small systems. Use `-march=native` to enable AVX2 auto-vectorization of the `add_scaled` and `apply_add` loops.

12.3 Dense vs. Sparse Crossover

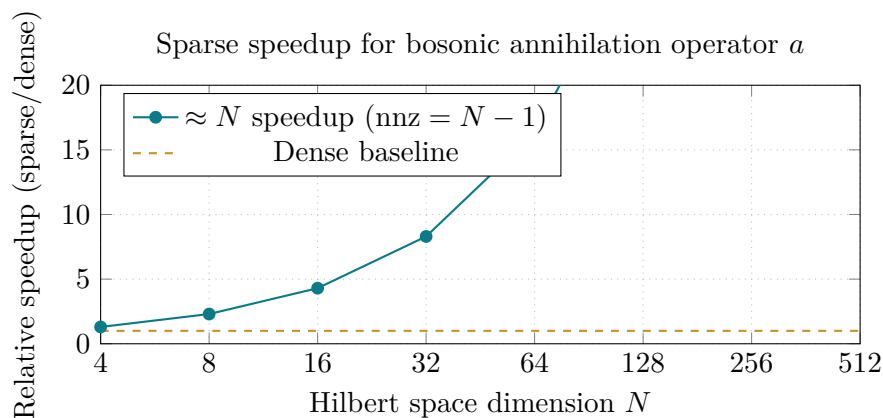


Figure 12.1: Theoretical speedup of sparse vs. dense matrix-vector product for the bosonic annihilation operator a (non-zero elements: $N - 1$). The crossover to practical benefit occurs around $N \approx 32$.

12.4 Multi-Threading Scaling

The ensemble is *embarrassingly parallel*: each trajectory is independent. With OpenMP enabled:

```
// In EnsembleConfig:
config.num_threads = std::thread::hardware_concurrency();
```

Each thread owns:

- One MCWFPropagator (and its scratch buffers).
- One ODE stepper (and its scratch buffers).
- One ObservableAccumulator (merged after each batch).

The shared read-only objects — `OpenSystem`, all operator matrices, observable definitions — are accessed concurrently without locks because they are immutable after construction.

OpenMP must be enabled at compile time (`-fopenmp`) and at CMake configuration time (`-DLIQUID_OPENMP=ON`). Setting `config.num_threads > 1` on a build without OpenMP has no effect; the simulation runs serially.

Extending LiQuID

13.1 Custom ODE Stepper

Any class satisfying the Stepper concept (section 6.1) can be used as a drop-in replacement:

```
1  // Minimal custom stepper: symplectic Euler (first-order)
2  class SymplecticEulerStepper {
3  public:
4      explicit SymplecticEulerStepper(Real dt) : dt_(dt) {}
5
6      template<typename RHSFn>
7      ode::StepResult try_step(Real t,
8                              const StateVector& psi_in,
9                              StateVector& psi_out,
10                             Real dt_suggested,
11                             Real t_max,
12                             RHSFn&& rhs_fn)
13      {
14          const Real dt = std::min({dt_suggested, dt_, t_max - t});
15          ensure_scratch(psi_in.size());
16
17          // Compute derivative:  $k = f(t, \psi_{in})$ 
18          k_.set_zero();
19          rhs_fn(t, psi_in, k_);
20
21          // Update:  $\psi_{out} = \psi_{in} + dt * k$ 
22          psi_out.copy_from(psi_in);
23          psi_out.add_scaled(k_, Scalar{dt, 0.0});
24
25          return ode::StepResult{true, dt, dt_, 0.0};
26      }
27
28      void reset(Real dt_initial) noexcept { dt_ = dt_initial; }
29
30  private:
31      void ensure_scratch(Dim n) {
```

```

32     if (k_.size() != n) k_ = StateVector(n);
33 }
34
35 Real      dt_;
36 StateVector k_;
37 };
38
39 // Use it with MCWFPPropagator:
40 MCWFPPropagator<DenseOpenSystem, SymplecticEulerStepper> prop(
41     &sys,
42     SymplecticEulerStepper(1e-4),
43     cfg);

```

13.2 Custom Allocation Policy

```

1  // Policy that preferentially runs more trajectories when
2  // the failure rate is below 5%, and falls back to uniform otherwise.
3  AllocatorPolicy my_policy = [seed = 42ULL](
4      const EnsembleSummary& summary) mutable -> AllocationDecision
5  {
6      const double failure_rate =
7          static_cast<double>(summary.trajectories_failed) /
8          std::max(summary.trajectories_completed, std::size_t{1});
9
10     AllocationDecision dec;
11     dec.action = AllocationAction::SpawnNew;
12
13     if (failure_rate < 0.05) {
14         // Low failure rate: use a deterministic seed sequence
15         dec.seed = seed++;
16     } else {
17         // High failure rate: fall back to random seed
18         dec.seed = std::hash<double>{}(summary.wall_time_elapsed);
19     }
20     return dec;
21 };
22
23 // Pass to Simulation::make() or EnsembleManager
24 auto sim = Simulation::make<DenseOpenSystem, ode::RK4Stepper>(
25     std::move(system),
26     std::move(observable_defs),
27     stopping_criteria,
28     ensemble_config,
29     my_policy);

```

Test Suite

14.1 Unit Tests

Table 14.1: Unit test executables and their scope.

Target	Scope
<code>test_rng</code>	xoshiro256** uniformity, period, seed independence
<code>test_linalg</code>	<code>StateVector</code> , <code>DenseOperator</code> : all methods and edge cases
<code>test_sparse</code>	CSR construction, <code>apply_add</code> , duplicate triplet handling
<code>test_opensystem</code>	<code>H_eff</code> precomputation, jump probabilities, <code>apply_jump</code>
<code>test_trajectory_state</code>	Factory postconditions, all seven invariants
<code>test_running_statistics</code>	Welford online mean/variance, single-sample edge case
<code>test_ensemble</code>	Convergence monitor, observable accumulator, parallel merge
<code>test_dopri45</code>	FSAL correctness, PI controller, step rejection
<code>test_ml_policy</code>	Feature extraction, MLP forward pass, SGD convergence
<code>test_parameter_sweep</code>	CSV output, factory invocation, OpenMP sweep

14.2 Validation Tests

Table 14.2: Physics validation tests and their analytical references.

Target	Reference
val_two_level_decay	$\langle \sigma_z \rangle = 2e^{-\gamma t} - 1$; passes within 3σ
val_driven_two_level	Driven qubit: steady-state Bloch vector vs. optical Bloch equations
val_ensemble_convergence	SEM $\propto 1/\sqrt{N}$: fit slope = -0.5 ± 0.05
val_cavity_qed	Jaynes–Cummings photon statistics: $g^{(2)}(0) < 1$ (antibunching)
val_smart_stopping	Early stopping triggers before <code>max_trajectories</code> when converged
val_ml_policy	ML policy SEM reduction rate $\geq$ uniform policy within 10%
val_parameter_sweep	Sweep monotonicity: $\langle \sigma_z \rangle_{ss}$ decreasing with γ

Run all tests with:

```
ctest --test-dir build --output-on-failure -j$(nproc)
```


Known Limitations and Roadmap

15.1 Known Limitations in v0.7.0

Table 15.1: Known limitations and planned resolution phases.

Limitation	Phase	Notes
Phase 1 jump location: no bisection for RK4	Phase 3 done	DOPRI45 path uses bisection
No BLAS dependency	Phase 3+	Manual loops; switch to BLAS for $N > 100$
<code>SimulationBuilder</code> : dense operators only	Phase 4	Use <code>Simulation::make()</code> for sparse
No Python bindings	Post-1.0	pybind11 wrapper planned
No GPU backend	Post-1.0	Explicit deferred decision
ML policy: seeds only, no rare-event biasing	Phase 6	Infrastructure present, biasing deferred

Mathematical Reference

A.1 Lindblad vs. GKSL Notation

Different communities use different conventions for eq. (2.1). LiQuID uses the following (natural units $\hbar = 1$):

$$\begin{aligned}\frac{d\hat{\rho}}{dt} &= -i[H, \hat{\rho}] + \sum_k \left(L_k \hat{\rho} L_k^\dagger - \frac{1}{2} L_k^\dagger L_k \hat{\rho} - \frac{1}{2} \hat{\rho} L_k^\dagger L_k \right) \\ &\equiv -i[H, \hat{\rho}] + \sum_k \mathcal{D}[L_k] \hat{\rho}\end{aligned}\tag{A.1}$$

where $\mathcal{D}[L] \hat{\rho} \equiv L \hat{\rho} L^\dagger - \frac{1}{2} \{L^\dagger L, \hat{\rho}\}$. Some references write $\gamma_k L_k$ with unit-norm L_k ; others absorb $\sqrt{\gamma_k}$ into L_k . LiQuID uses the latter convention: $L_k = \sqrt{\gamma_k} \tilde{L}_k$ where $\tilde{L}_k$ is dimensionless.

A.2 Standard Error of the Mean

For M independent MCWF samples $x_1, \dots, x_M$ of an observable, LiQuID computes:

$$\hat{\mu} = \frac{1}{M} \sum_{m=1}^M x_m \tag{A.2}$$

$$\hat{s}^2 = \frac{1}{M-1} \sum_{m=1}^M (x_m - \hat{\mu})^2 \tag{A.3}$$

$$\widehat{\text{SEM}} = \frac{\hat{s}}{\sqrt{M}} \tag{A.4}$$

$$\varepsilon_{\text{rel}} = \frac{\widehat{\text{SEM}}}{|\hat{\mu}|} \tag{A.5}$$

The stopping criterion $\varepsilon_{\text{rel}} < \varepsilon_{\text{target}}$ is applied to the worst-case observable across all channels. The variance $\hat{s}^2$ is computed via Welford's online algorithm (implemented in `RunningStatistics`) to avoid catastrophic cancellation.

API Quick Reference

B.1 Builder Method Summary

Table B.1: Complete SimulationBuilder method reference.

Method	Type	Sets field
.hamiltonian(H)	DenseOperator	H_
.collapse_operator(L)	DenseOperator	adds to L_list_
.observe(name, 0)	string, DenseOperator	adds to obs_
.observe(name, fn)	string, ObservableFn	adds to obs_
.dt(h)	Real	cfg.dt_initial
.solver(type)	SolverType	solver_
.atol(e)	Real	cfg.atol
.rtol(e)	Real	cfg.rtol
.seed(s)	Seed	ens_cfg.global_seed
.num_threads(n)	size_t	ens_cfg.num_threads
.stop_at_sem(e)	Real	sc.target_rel_sem
.min_trajectories(n)	size_t	sc.min_trajectories
.max_trajectories(n)	size_t	sc.max_trajectories
.max_wall_time(s)	Real	sc.max_wall_seconds
.diagnostic_level(l)	DiagnosticLevel	ens_cfg.diag_level
.build()	→ Simulation	validates + constructs

Bibliography

- [1] J. Dalibard, Y. Castin, and K. Mølmer, “Wave-function approach to dissipative processes in quantum optics,” *Phys. Rev. Lett.* **68**, 580 (1992).
- [2] R. Dum, P. Zoller, and H. Ritsch, “Monte Carlo simulation of the atomic master equation for spontaneous emission,” *Phys. Rev. A* **45**, 4879 (1992).
- [3] K. Mølmer, Y. Castin, and J. Dalibard, “Monte Carlo wave-function method in quantum optics,” *J. Opt. Soc. Am. B* **10**, 524 (1993).
- [4] H. Carmichael, *An Open Systems Approach to Quantum Optics*. Springer, Berlin, 1993.
- [5] C. W. Gardiner and P. Zoller, *Quantum Noise*, 3rd ed. Springer, Berlin, 2004.
- [6] J. R. Dormand and P. J. Prince, “A family of embedded Runge-Kutta formulae,” *J. Comput. Appl. Math.* **6**, 19–26 (1980).
- [7] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I*, 2nd ed. Springer, Berlin, 1993.
- [8] K. Gustafsson, “Control theoretic techniques for stepsize selection in explicit Runge-Kutta methods,” *ACM Trans. Math. Softw.* **17**, 533–554 (1991).
- [9] D. Blackman and S. Vigna, “Scrambled linear pseudorandom number generators,” *ACM Trans. Math. Softw.* **47**, 1–32 (2021). [arXiv:1805.01407](#).
- [10] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics* **4**, 419–420 (1962).
- [11] G. Lindblad, “On the generators of quantum dynamical semigroups,” *Commun. Math. Phys.* **48**, 119–130 (1976).
- [12] V. Gorini, A. Kossakowski, and E. C. G. Sudarshan, “Completely positive dynamical semigroups of N -level systems,” *J. Math. Phys.* **17**, 821–825 (1976).